SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Name: Mihiran Hanumat PRN: 24070126113
Academic Year: 2026-2027 Sem: V
Course: Data Visualization Lab Course Code: TE7663

Assignment Practical No. 4

Aim

To use Power BI's Power Query editor and DAX to clean, reshape, merge and analyse the Adventure Works sales dataset , covering grouping, pivoting/unpivoting, appending, merging, sales analysis measures, and conditional-column logic for deeper, dashboard-ready insights.

Procedure Followed

The Adventure Works Raw Data was loaded into Power BI, with the three yearly sales files combined into one query and the lookup tables (Product, Category, Subcategory, Territory, Customer) loaded individually. Relationships were built in Model view, and a flat working table was created by merging the sales data with the relevant lookups. All Part A tasks were built as Power Query Group By operations on this working table (with the corresponding measure/card shown on the report canvas), Part B/C/D reshape and combine this data using Pivot, Unpivot, Append and Merge, and Part B (Sales Analysis) plus the Advanced tasks layer DAX measures and Conditional Columns on top for the final dashboard.

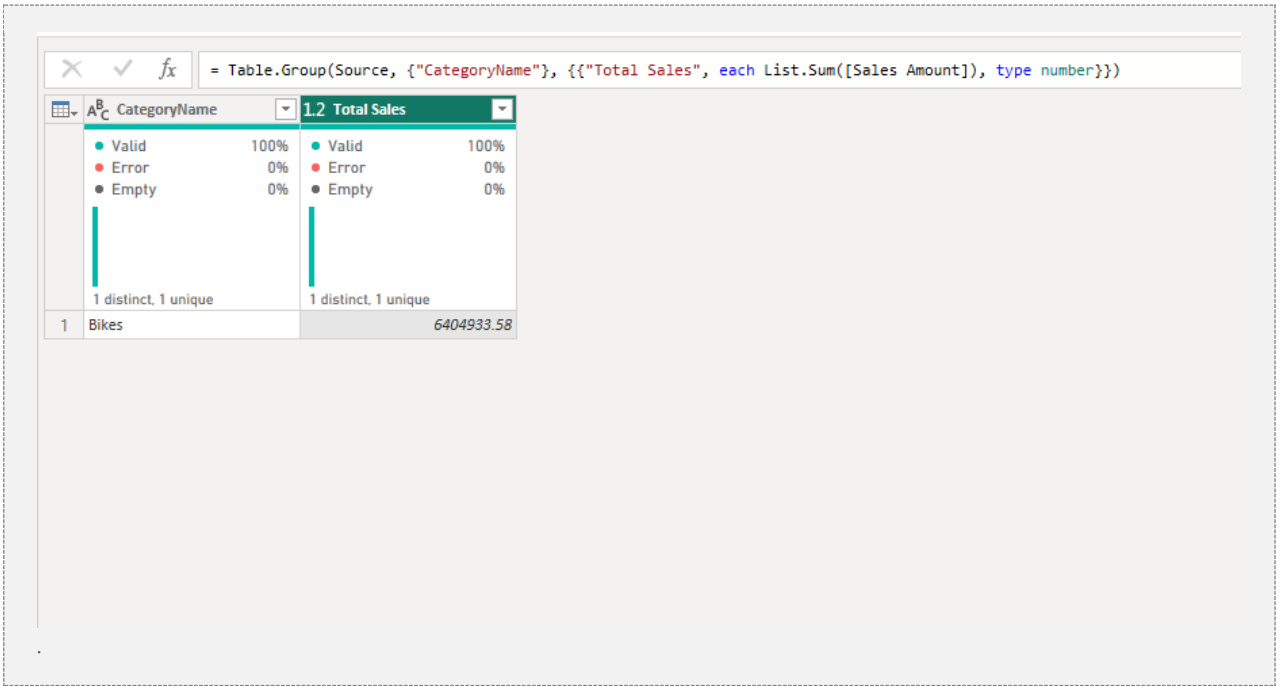
Part A: Power Query Operations

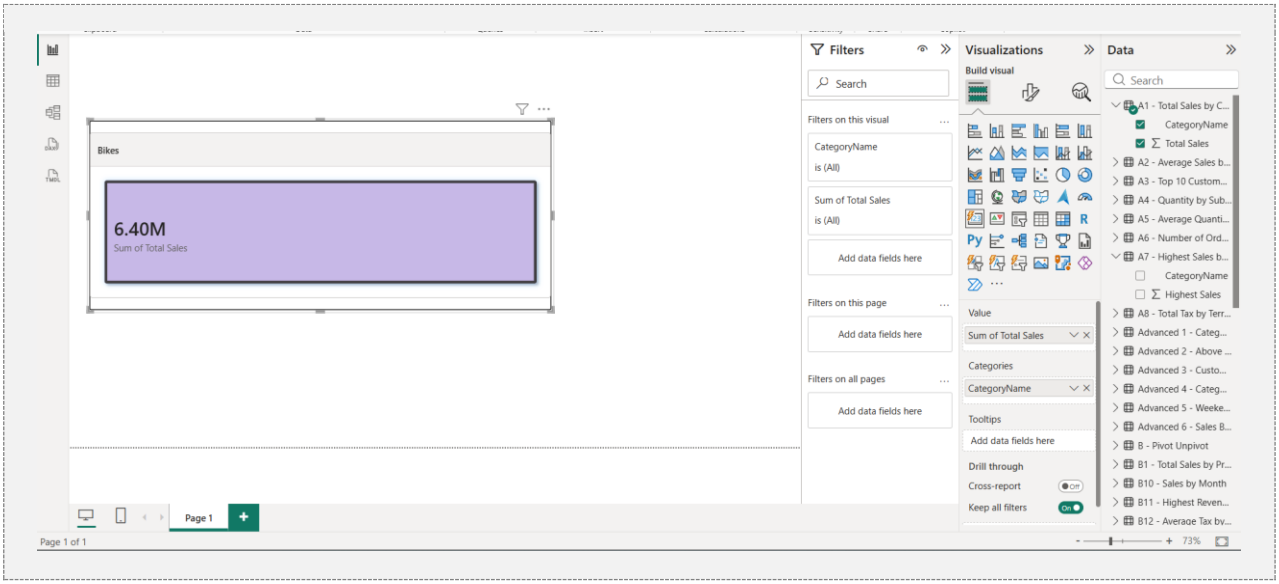
A. Grouping

Grouping (Group By) is Power Query's core aggregation tool — it collapses many rows into one row per group by applying Sum, Average, Count, Min or Max to a chosen column, similar to a SQL GROUP BY.

1. Find the total sales amount for each product category.
Grouping is Power Query's equivalent of a GROUP BY — it collapses many transaction rows into one row per category by applying an aggregation to a numeric column.

1. Select the CategoryName column on the merged sales table.
2. Home > Group By, set aggregation to Sum on Sales Amount.
3. Rename the output column to 'Total Sales'.

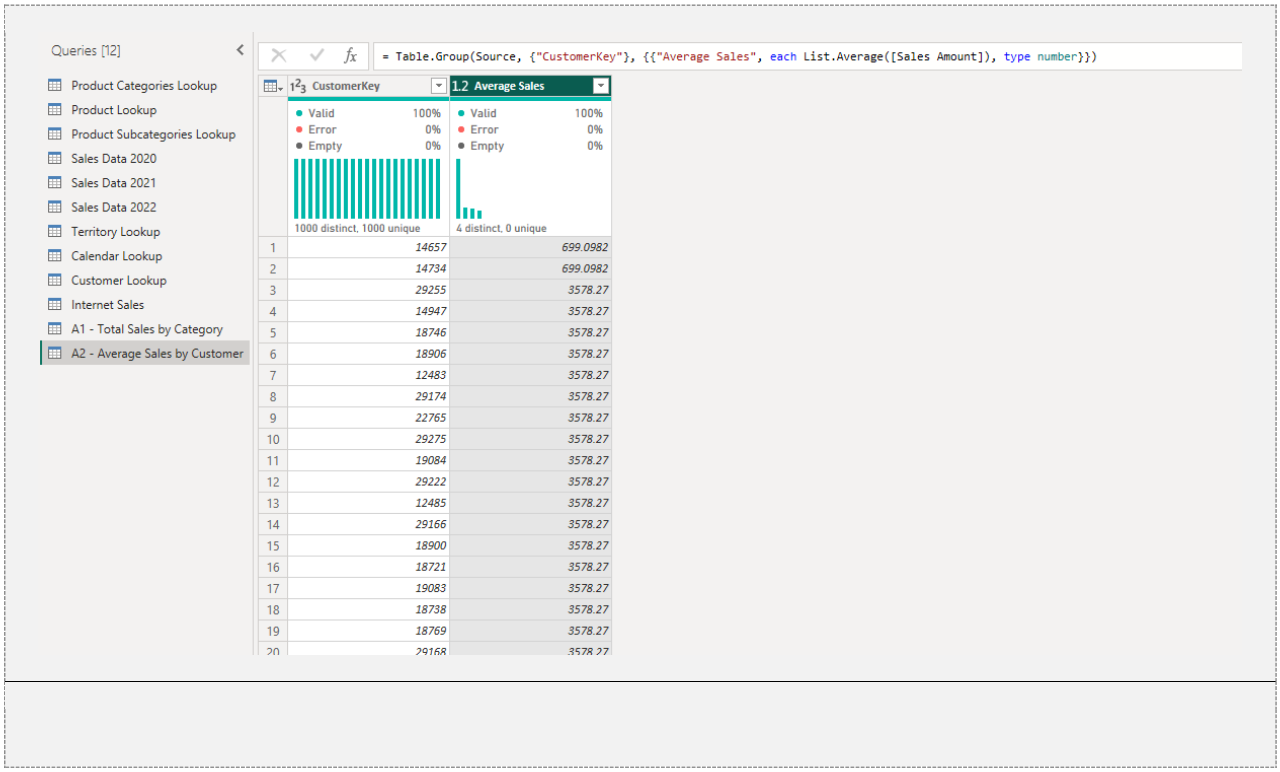


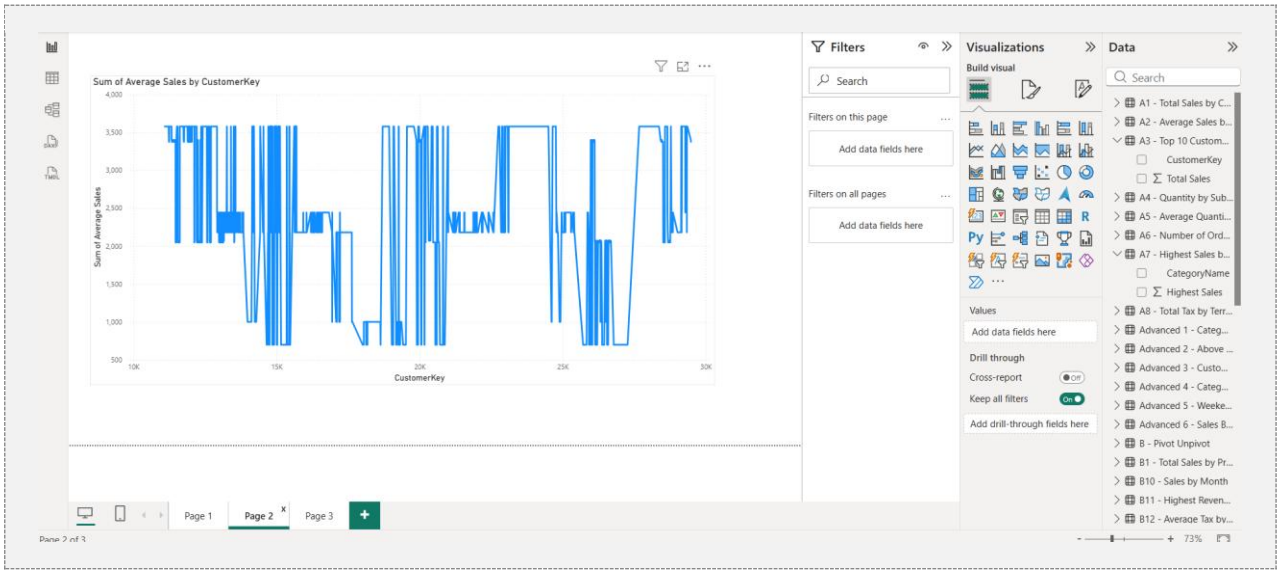


2. Calculate the average sales amount for each customer.

Switching the aggregation type from Sum to Average turns the same Group By operation into a per-customer spend benchmark instead of a total.

1. Select CustomerKey as the grouping column.
2. Group By with aggregation = Average on Sales Amount.
3. Rename the result column to 'Average Sales'.

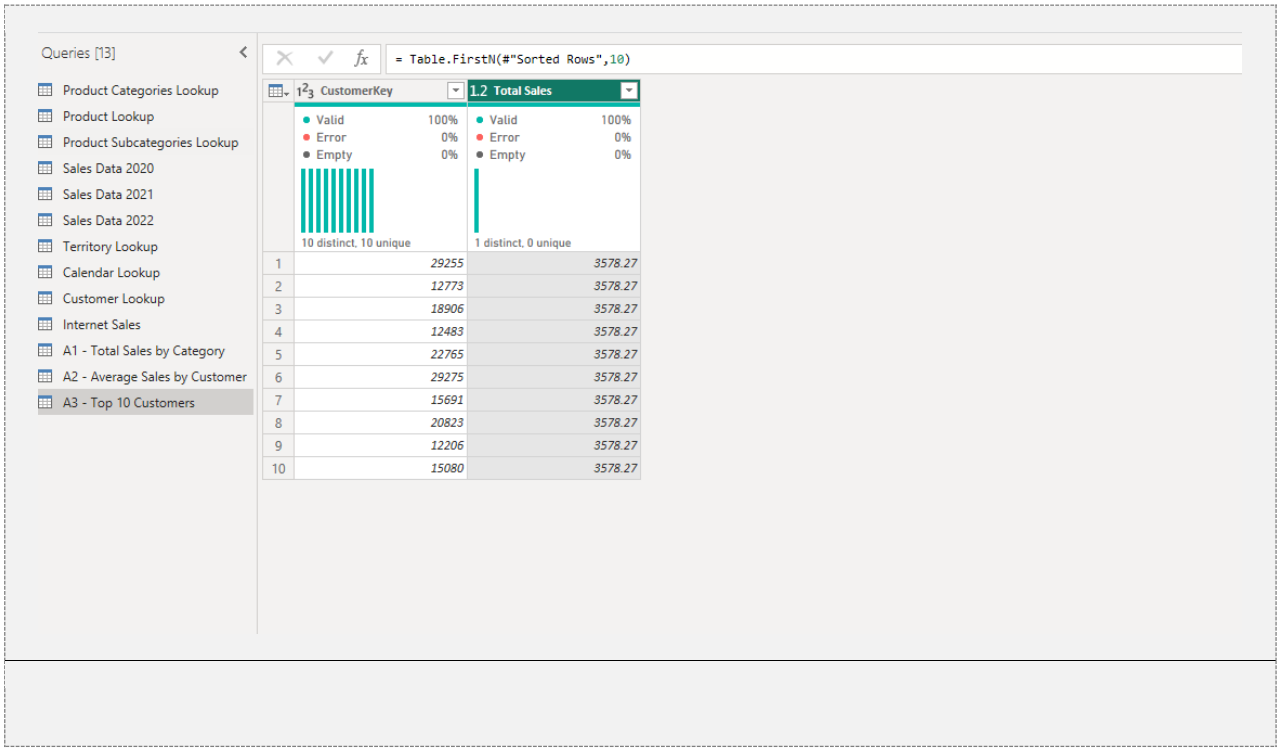


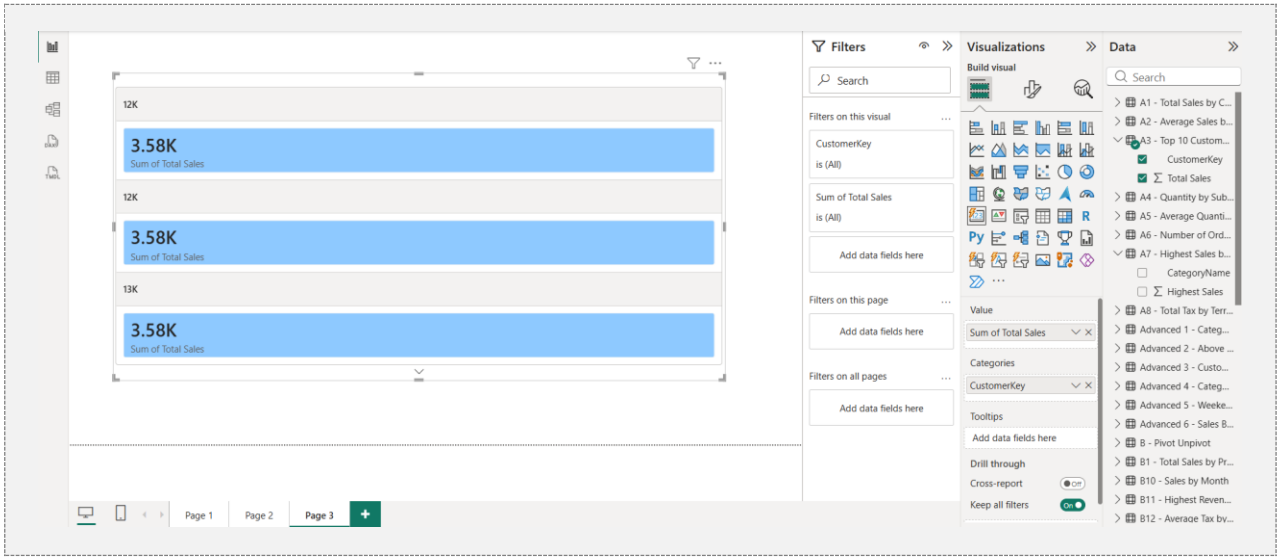


3. Identify the top 10 customers based on total sales.

Once customers are grouped and totalled, a Top N filter on the visual narrows the full list down to just the highest performers — much easier to read on a dashboard than a 17,000-row table.

1. Group Sales Analysis by CustomerKey, Sum of Sales Amount.
2. Sort the resulting table descending by Total Sales.
3. Keep only the first 10 rows (Table.FirstN) or apply a Top N visual filter.

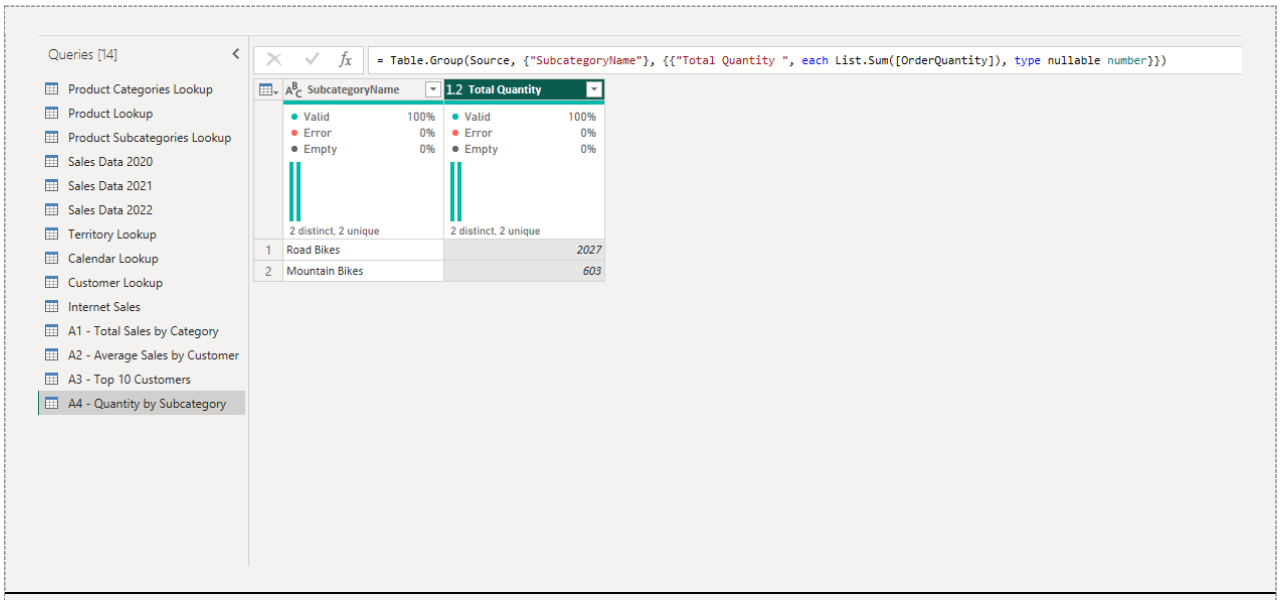


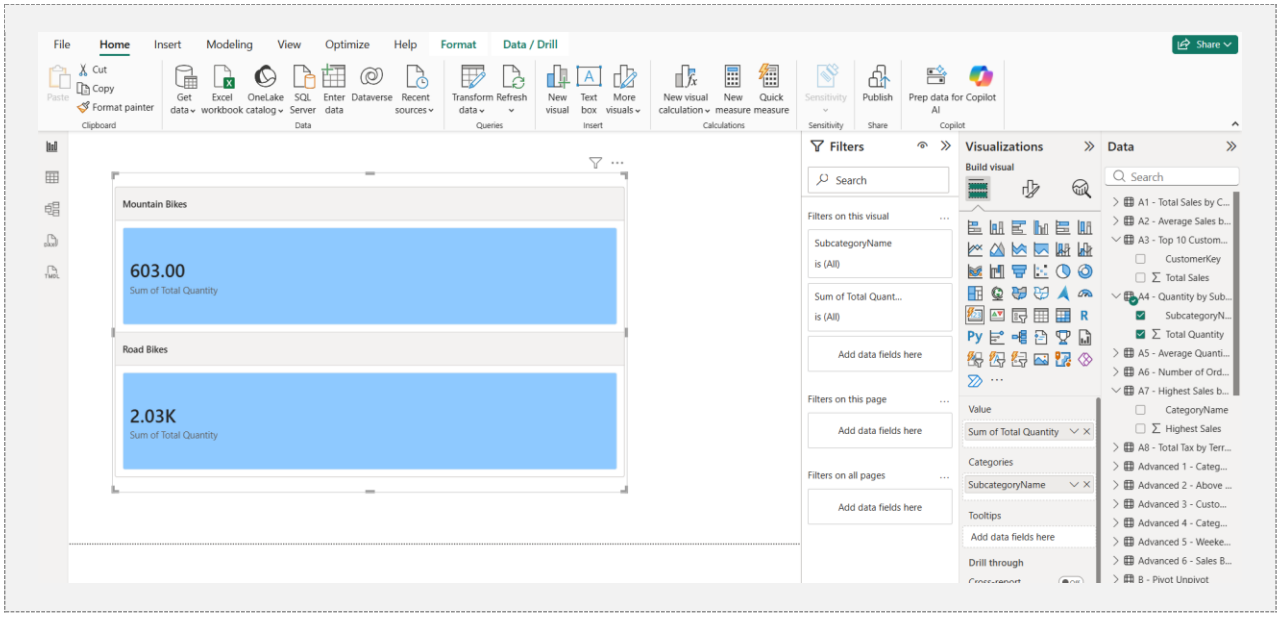


4. Find the total quantity sold for each product subcategory.

The same Group By pattern works on any numeric column — here it's applied to Order Quantity instead of Sales Amount, giving a units-moved view rather than a revenue view.

1. Select ProductSubcategoryKey / SubcategoryName as the grouping column.
2. Group By with aggregation = Sum on Order Quantity.
3. Rename to 'Total Quantity Sold'.

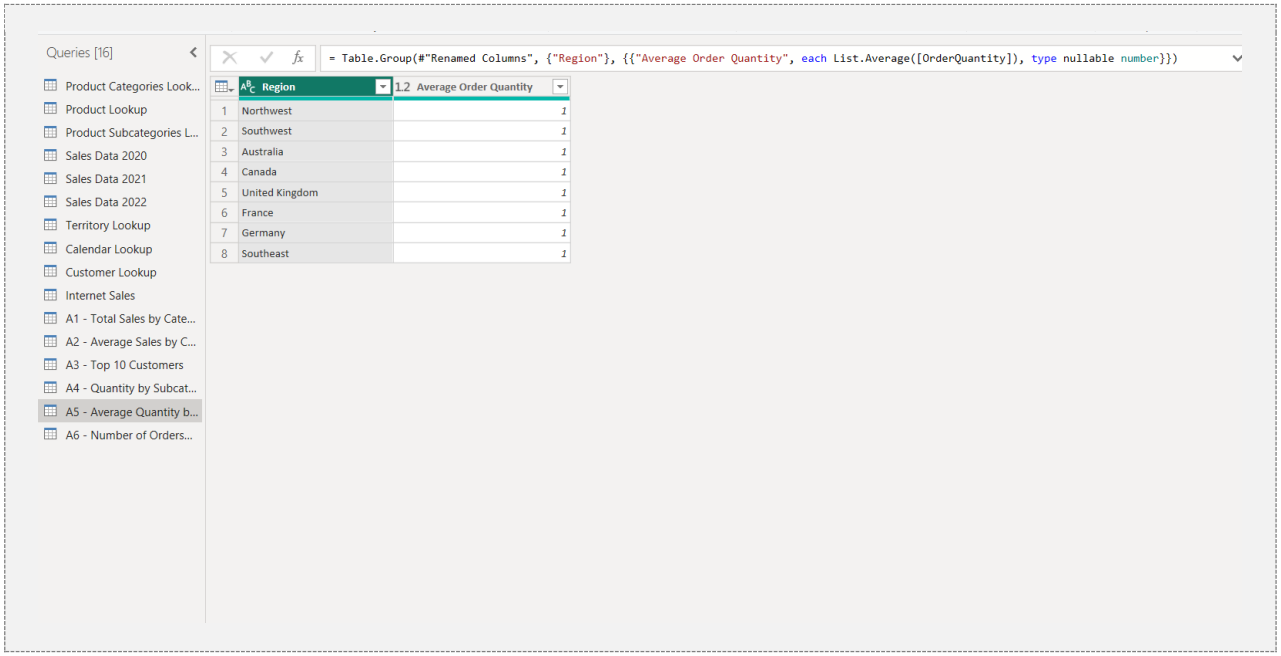


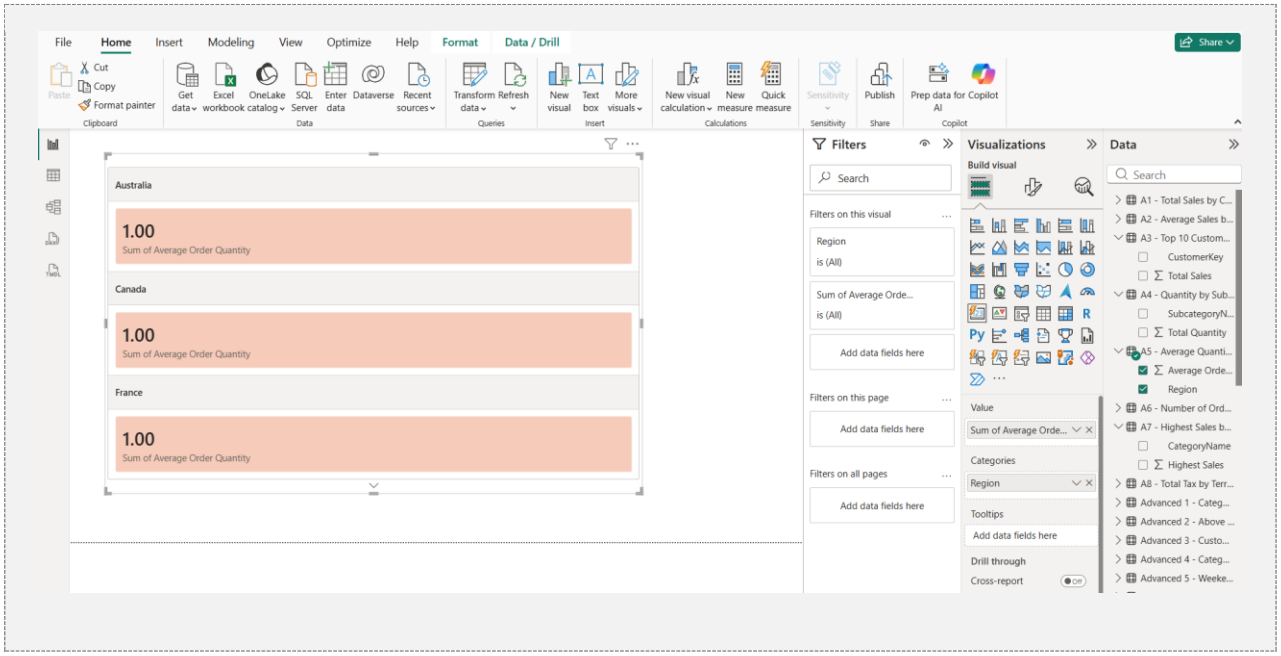


5. Determine the average order quantity for each sales territory.

Average Order Quantity tells you the typical basket size per region, which is a different signal from total revenue — a territory can have high sales purely from volume of small orders, or from fewer large ones.

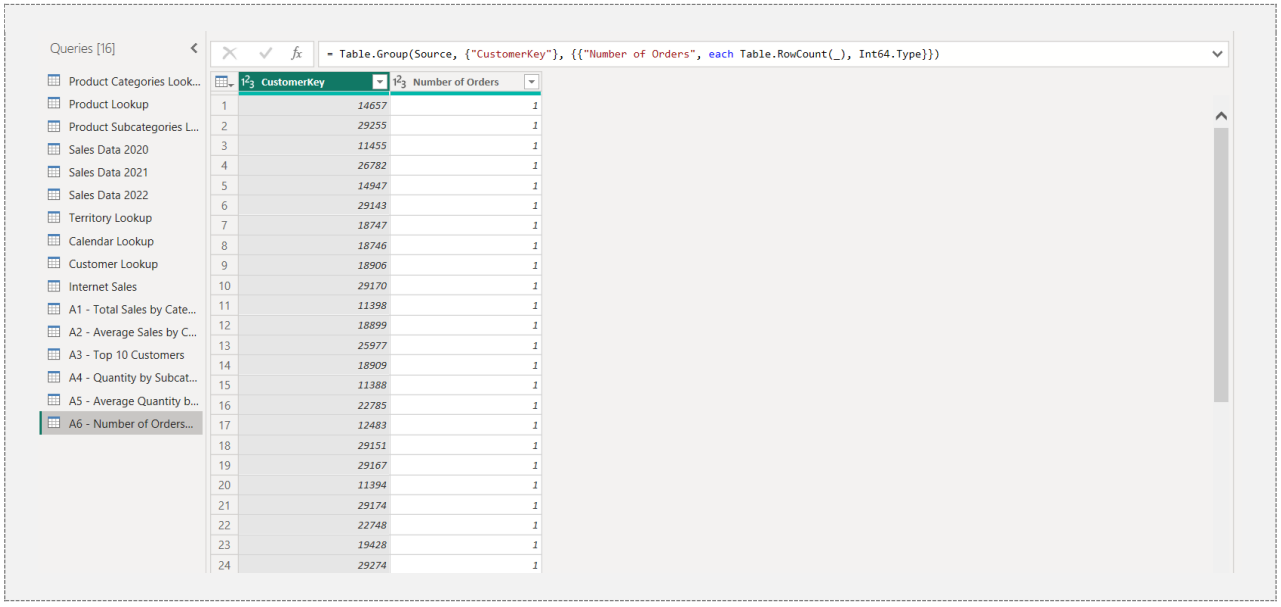
1. Merge the sales table with the Territory lookup on TerritoryKey to bring in Region.
2. Group By Region, aggregation = Average on Order Quantity.
3. Rename the output column to 'Avg Order Quantity'.

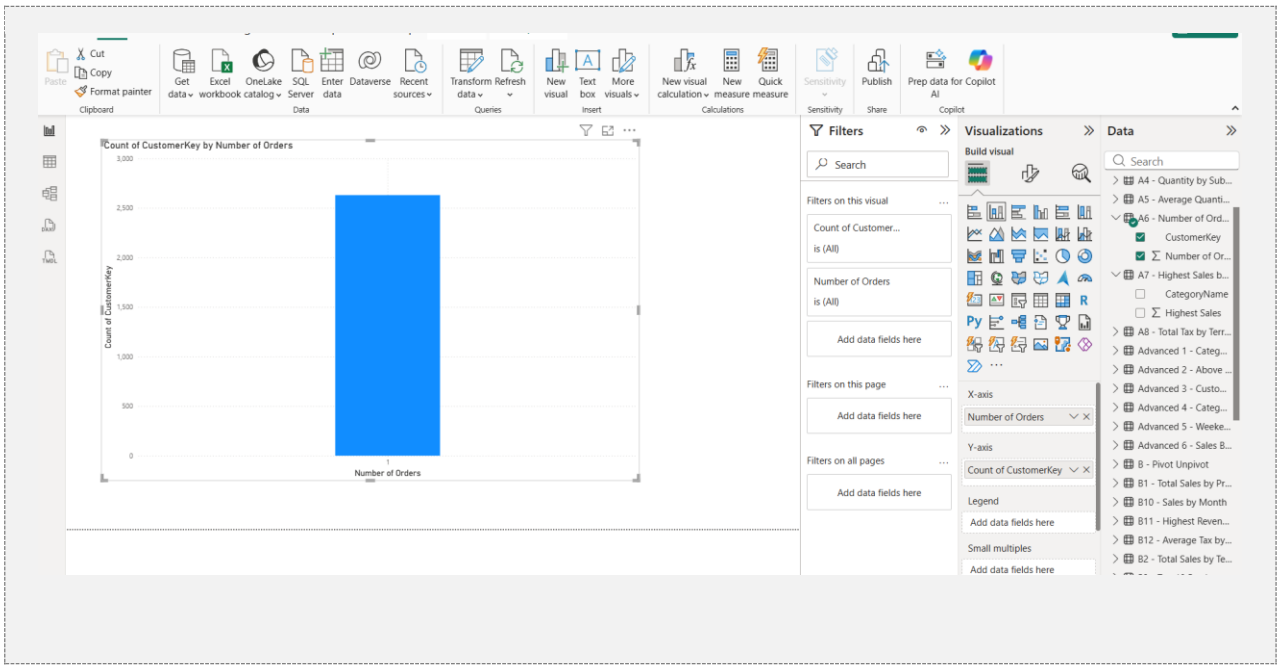




6. Count the number of orders placed by each customer.
Because the sales table is at line-item level (one row per product in an order), a plain row count would overstate order volume — Count Distinct on OrderNumber avoids double-counting multi-item orders.

1. Select CustomerKey as the grouping column.
2. Group By, aggregation = Count Distinct Rows on OrderNumber.
3. Rename the output column to 'Order Count'.

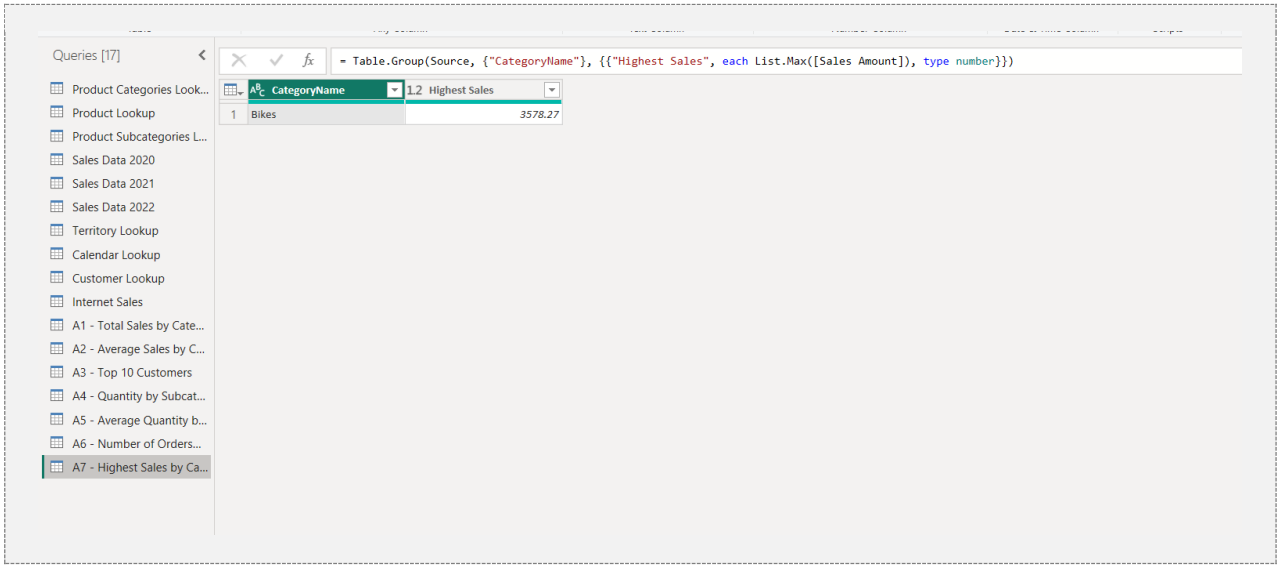


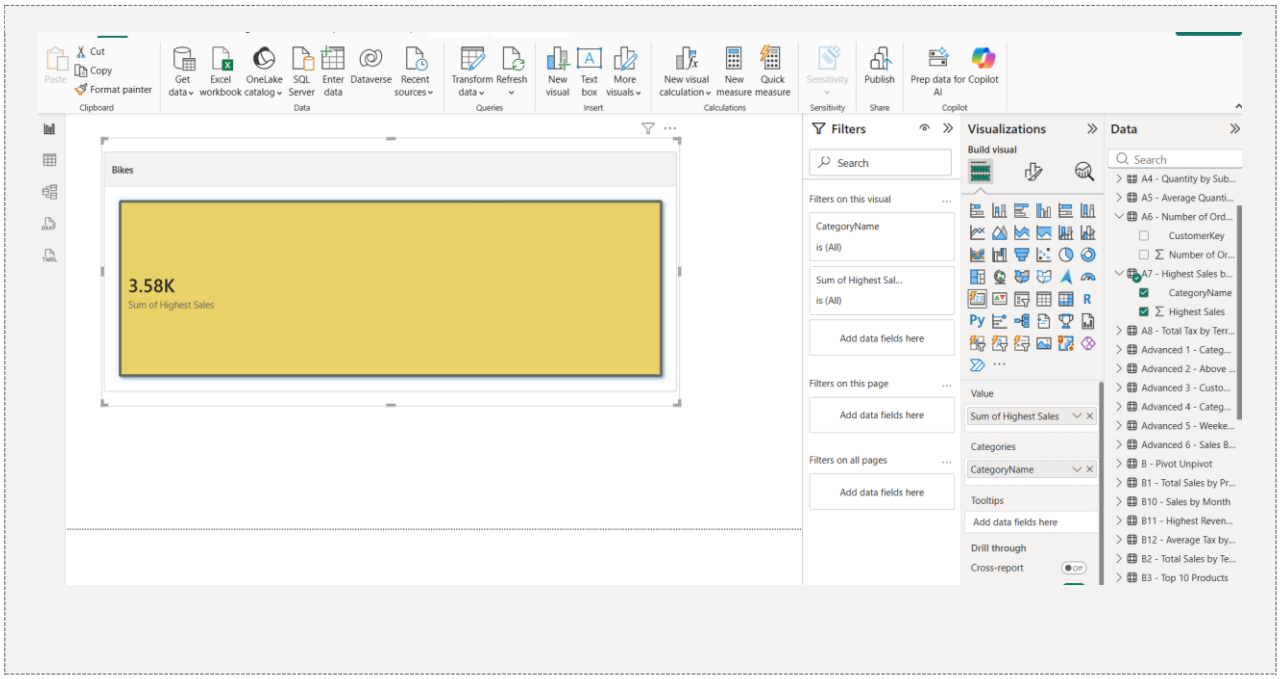


7. Find the highest sales amount recorded for each product category.

Max, unlike Sum or Average, surfaces the single biggest transaction within each group — handy for spotting outlier orders or premium purchases rather than overall trend.

1. Group by CategoryName (after merging in the category lookup chain).
2. Group By, aggregation = Max on Sales Amount.
3. Rename the output column to 'Max Sales Amount'.

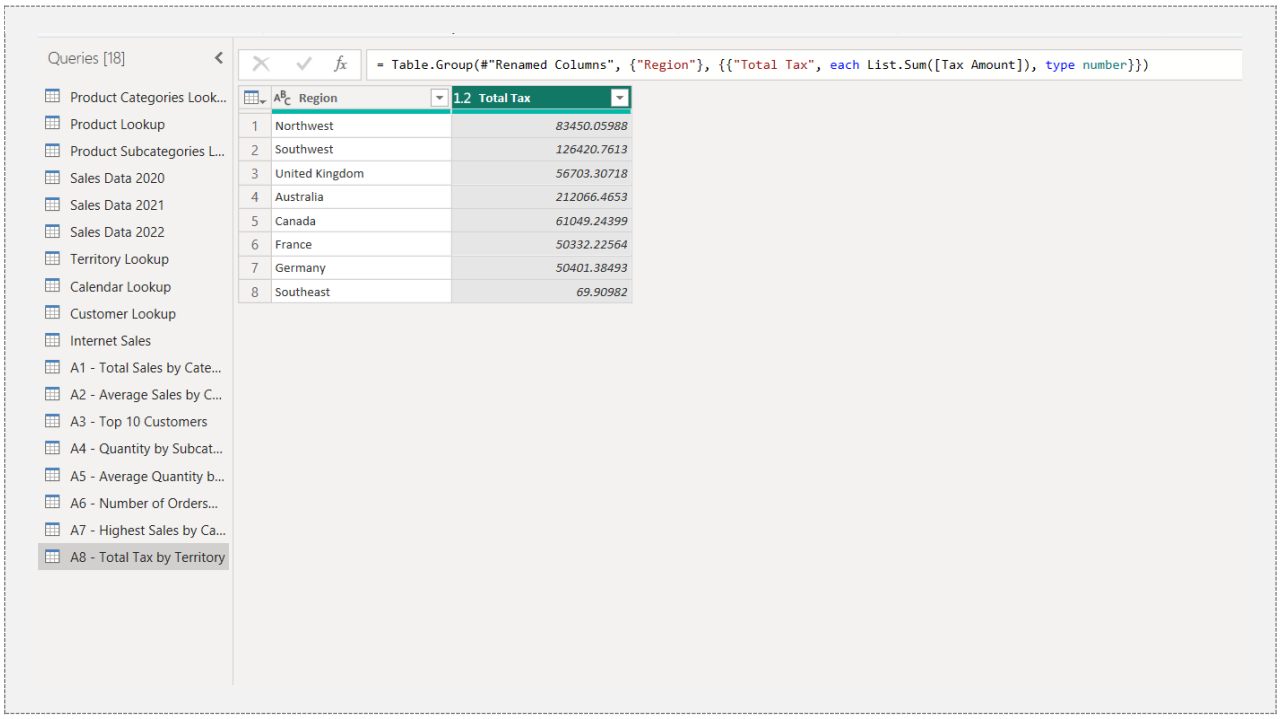


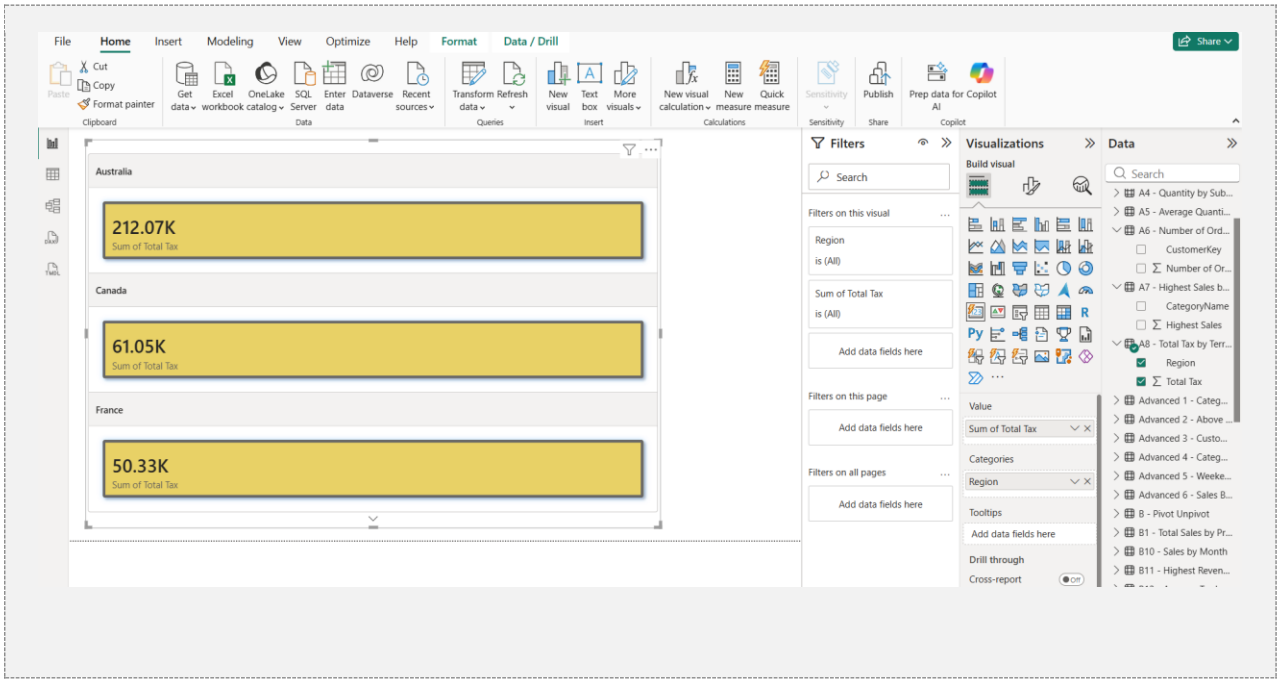


8. Calculate the total tax amount collected for each sales territory.

Tax Amount is typically derived as a percentage of Sales Amount (this dataset assumes the standard 8% convention used in this AdventureWorks tutorial series), then summed per region to show where the biggest tax burden sits.

1. Add a calculated Tax Amount column = Sales Amount * 0.08 if not already present.
2. Group By Region, aggregation = Sum on Tax Amount.
3. Rename the output column to 'Total Tax Amount'.



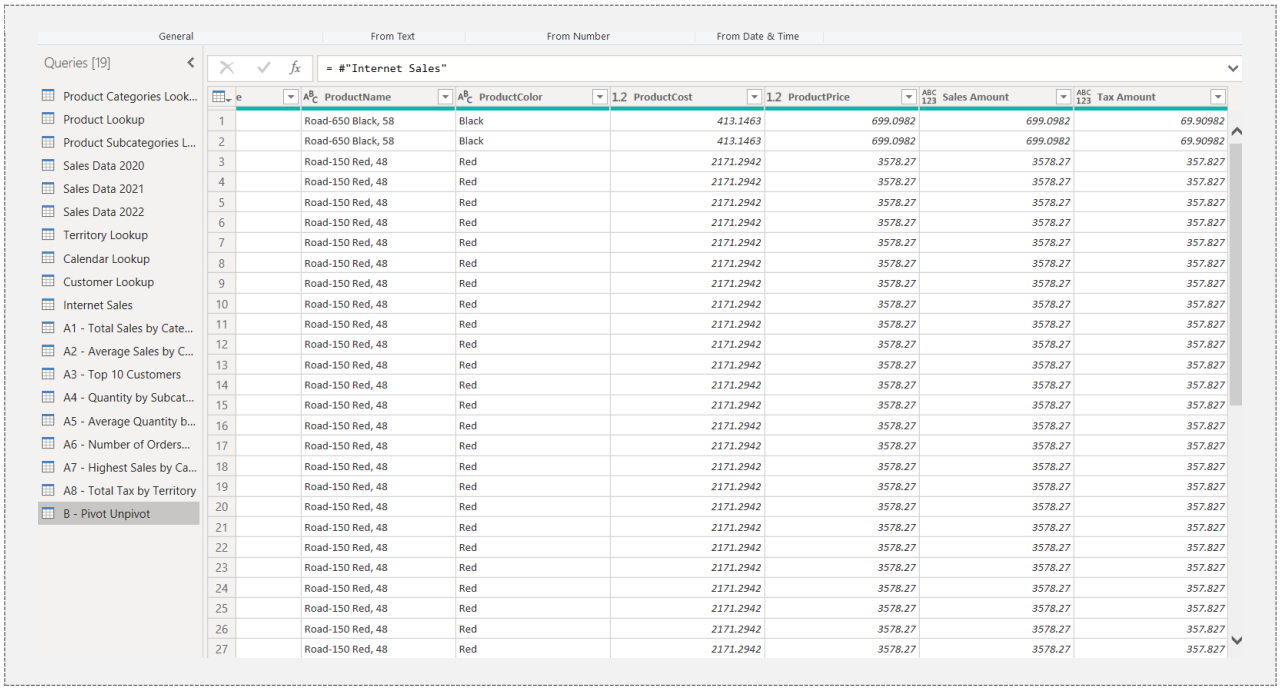


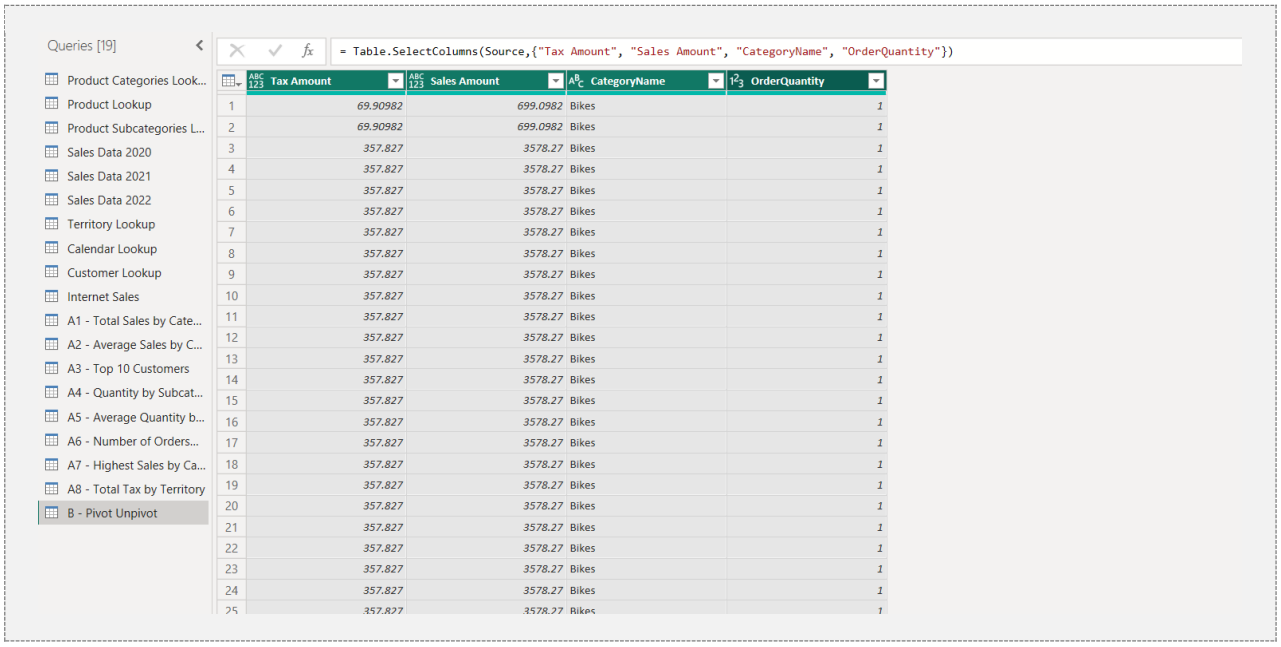
B. Pivoting and Unpivoting

Pivot and Unpivot reshape a table between 'wide' (one column per metric) and 'long/narrow' (one row per metric) formats. Long format is generally what Power BI visuals and slicers expect, while wide format is easier for humans to read as a table.

1. Create a table containing Product Category, Sales Amount, Tax Amount, and Order Quantity.
Before reshaping data with Pivot/Unpivot it helps to isolate just the columns involved, so the transformation is easy to follow and doesn't get lost among unrelated fields.

1. Start from the fully merged Sales Analysis table.
2. Select Product Category, Sales Amount, Tax Amount and Order Quantity.
3. Right-click > Remove Other Columns to keep only these four.

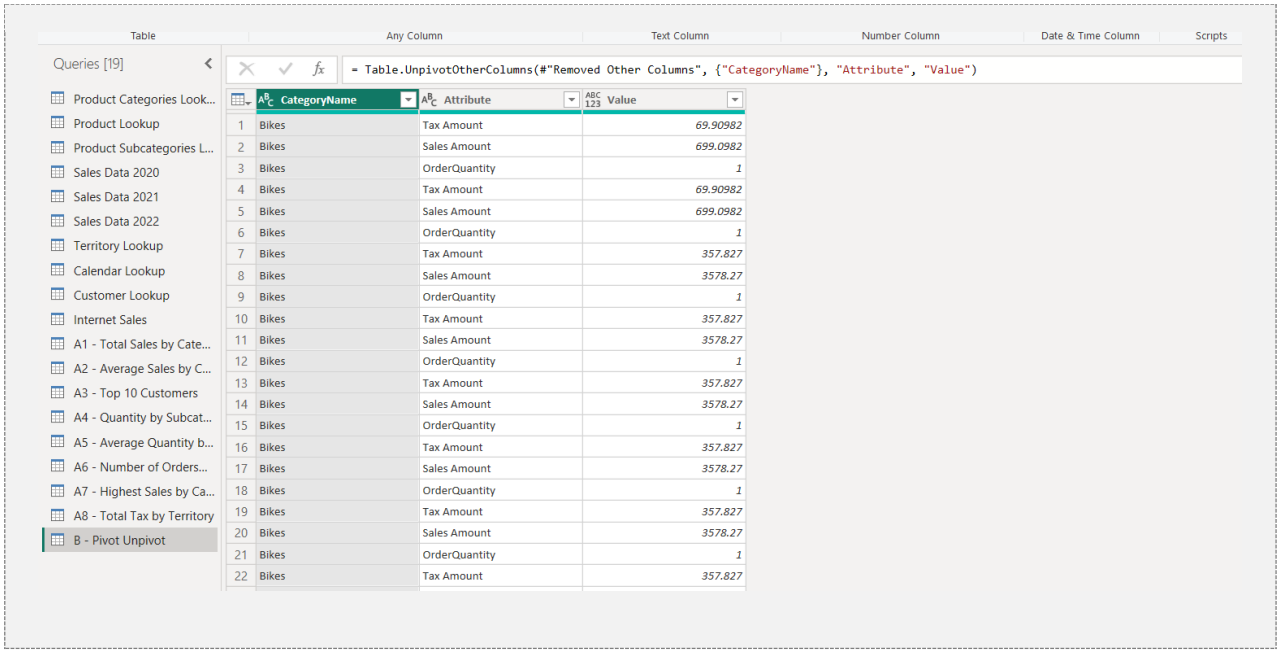




2. Unpivot the numerical columns (Sales Amount, Tax Amount, Order Quantity) into Metric and Value columns.

Unpivot turns a wide table (one column per metric) into a long/narrow table (one row per metric per category) — this shape is what most dynamic Power BI visuals and slicers expect.

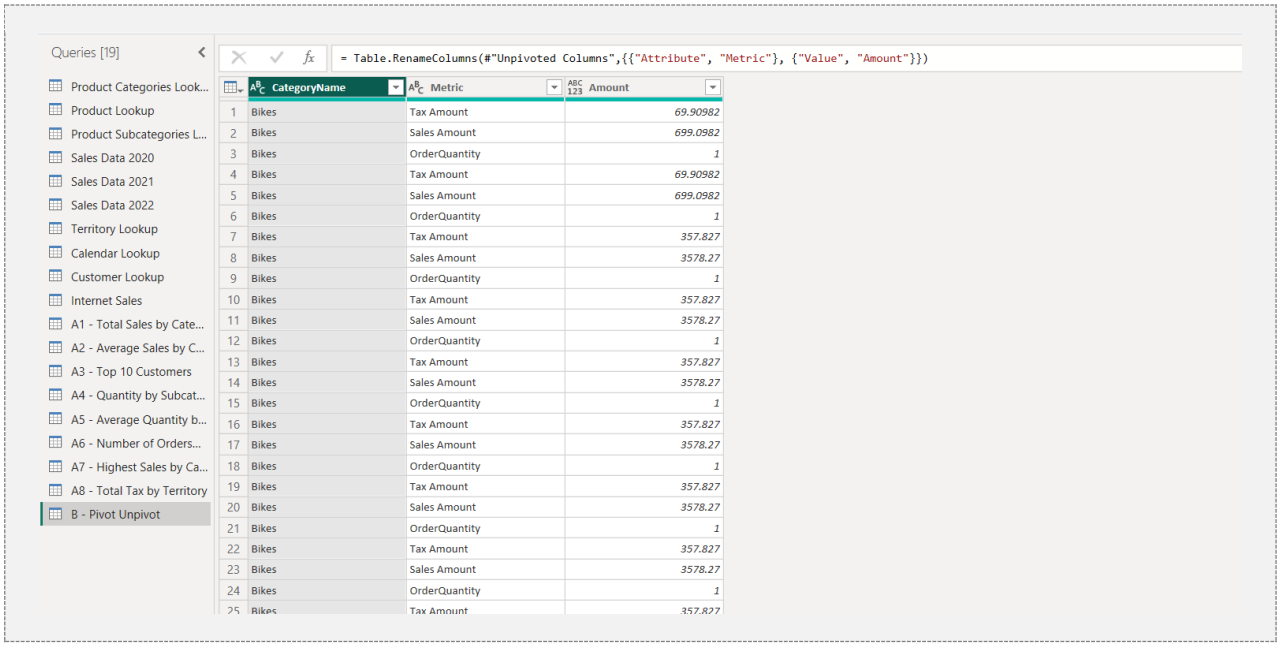
1. Select the Sales Amount, Tax Amount and Order Quantity columns together.
2. Right-click the column headers > Unpivot Columns.
3. Power Query creates two new columns, Attribute and Value, holding the metric name and its number.



3. Rename the generated columns: Attribute to Metric, Value to Amount.

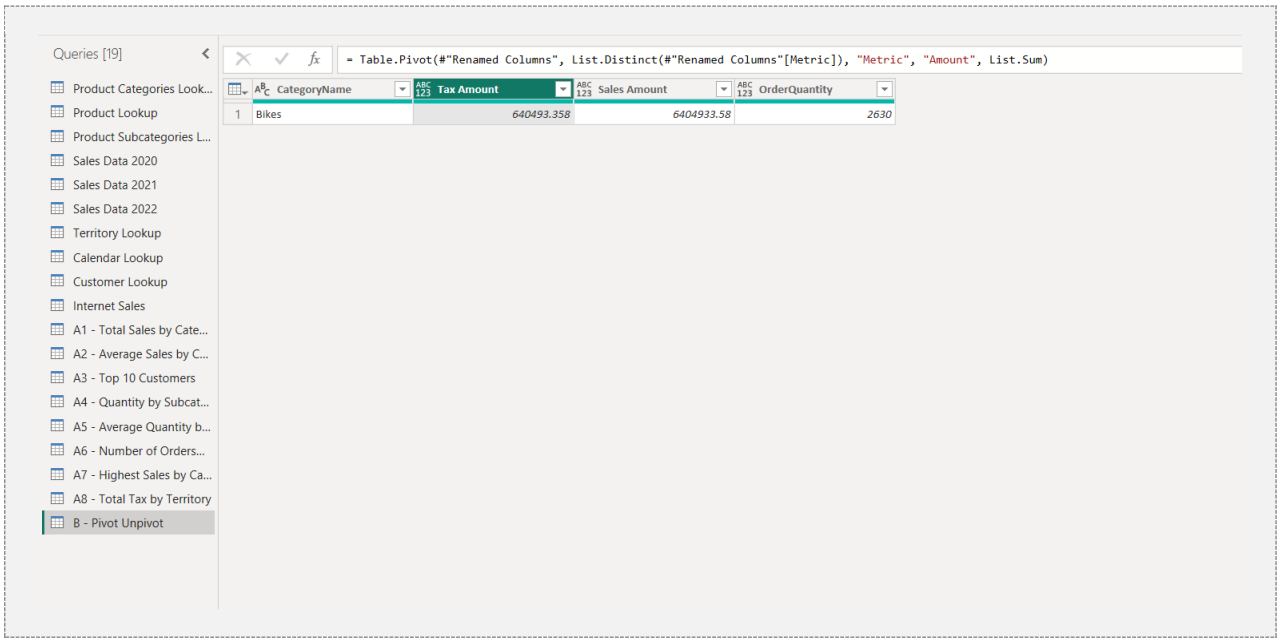
The default Attribute/Value names from the Unpivot step are generic — renaming them to Metric/Amount makes the model self-documenting for anyone using it as a slicer field.

1. Double-click the Attribute column header, type 'Metric'.
2. Double-click the Value column header, type 'Amount'.
3. Confirm the Applied Steps pane shows a 'Renamed Columns' step.



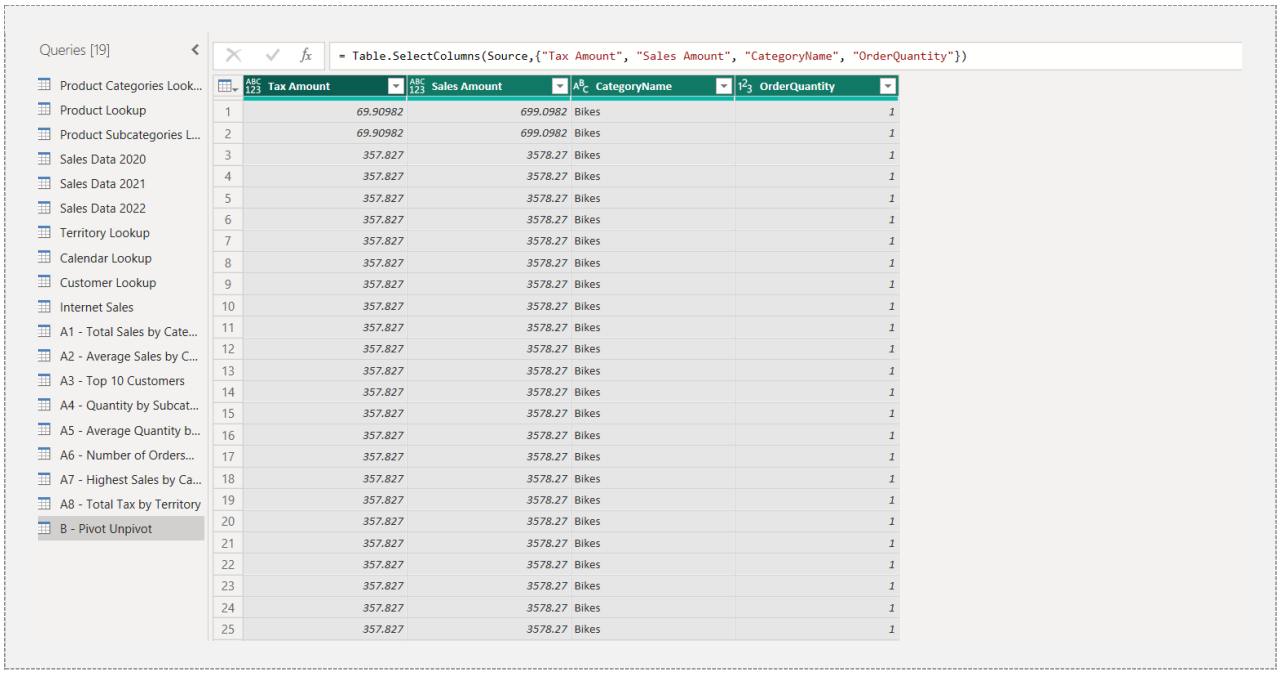
4. Pivot the Metric column back to its original structure.
Pivot is the reverse of Unpivot — it takes the long/narrow table and spreads the Metric values back out into separate columns, restoring the original wide shape.

1. Select the Metric column.
2. Transform > Pivot Column.
3. Choose Amount as the Values Column, with Sum (or Don't Aggregate) as the aggregation.



5. Verify that the pivoted table matches the original data.
A round-trip check (unpivot then pivot) should return you to the same shape and values you started with — comparing row/column counts is a quick sanity test that nothing was lost in the transformation.

1. Compare the row count and column count of the re-pivoted table against the original staging table from B1.
2. Spot-check a few sample values for the same category.
3. Confirm they match exactly.

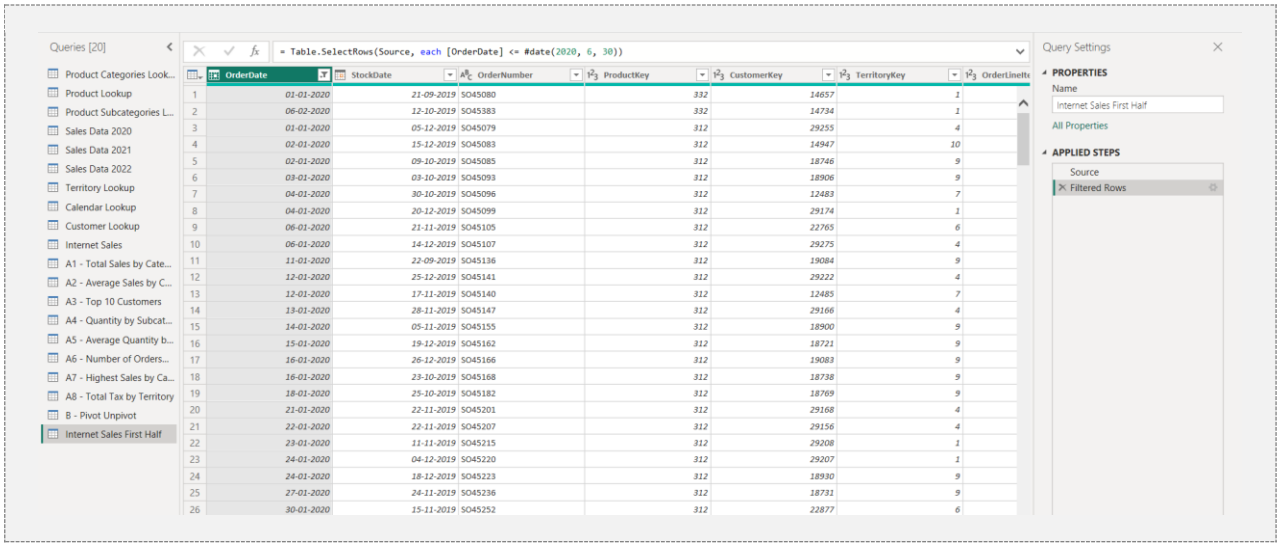


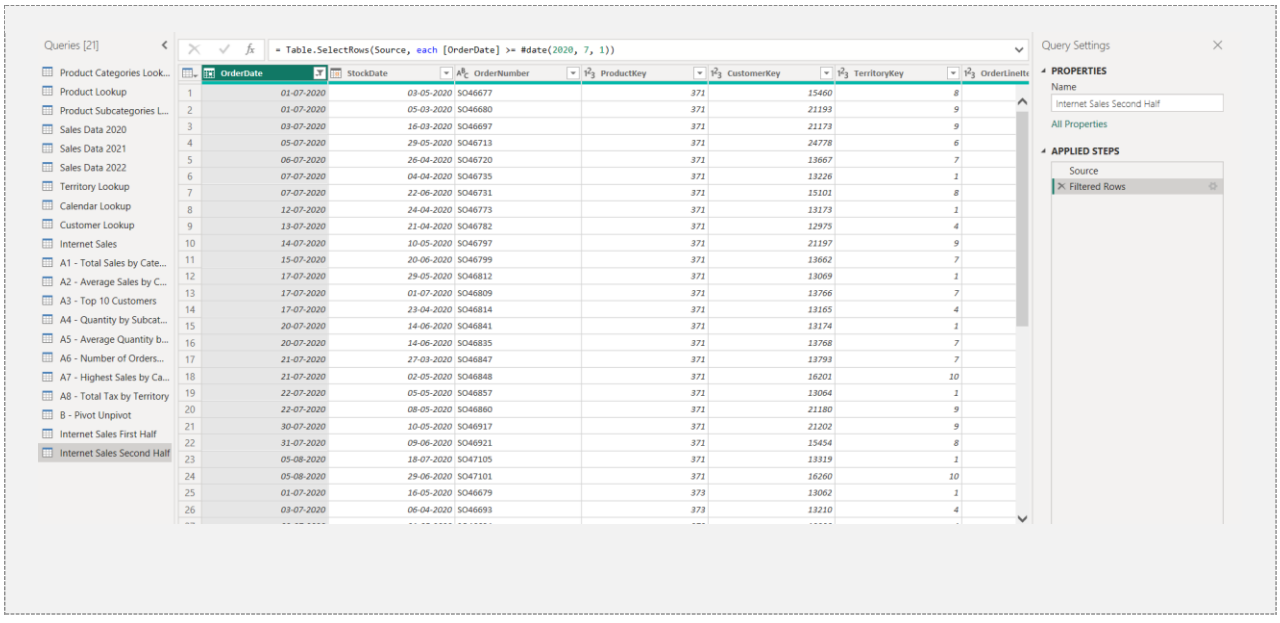
C. Append Queries

Append stacks the rows of two or more queries with matching columns into a single combined table — the Power Query equivalent of a SQL UNION ALL, commonly used to combine data split across files or time periods.

1. Create two queries: Internet Sales (First Half) and Internet Sales (Second Half).
Splitting a table by date is a common pattern for testing Append — it simulates two separate data sources (e.g. two exports) that need to be stacked back together.

1. Duplicate the sales query twice.
2. Filter one copy to Order Date/Year in the first half (e.g. 2020-2021) and name it 'Internet Sales (First Half)'.
3. Filter the other copy to the remaining years and name it 'Internet Sales (Second Half)'.

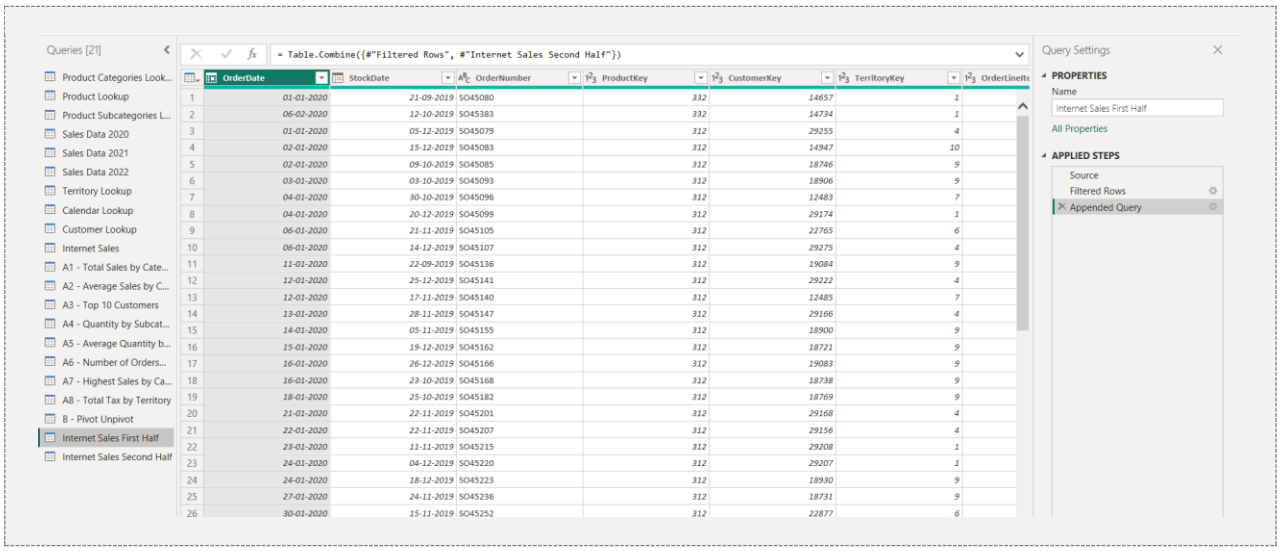




2. Append the two queries into a single table.

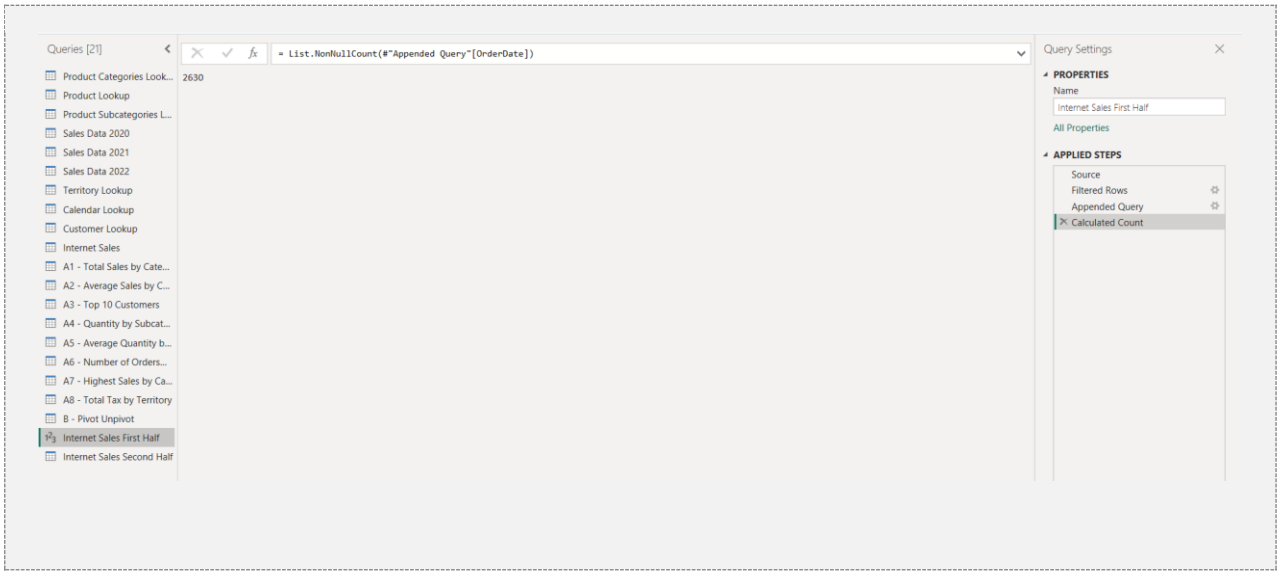
Append stacks rows from two (or more) queries with the same columns into one combined table — this is the Power Query equivalent of a SQL UNION ALL.

1. Select Internet Sales (First Half).
2. Home > Append Queries > Append Queries as New.
3. Choose Internet Sales (Second Half) as the table to append.



3. Verify the total number of records after appending.
The appended table's row count should equal the sum of the two source queries' row counts — any mismatch would indicate duplicate or dropped rows during the append.

1. Check the row count shown in the status bar / Applied Steps for the appended query.
2. Compare it against the sum of First Half + Second Half row counts.



D. Merge Queries

Merge is Power Query's join operation, similar to a SQL JOIN or Excel VLOOKUP — it attaches a nested table of matching rows from a second query based on a shared key, which is then Expanded to pull specific fields into flat columns.

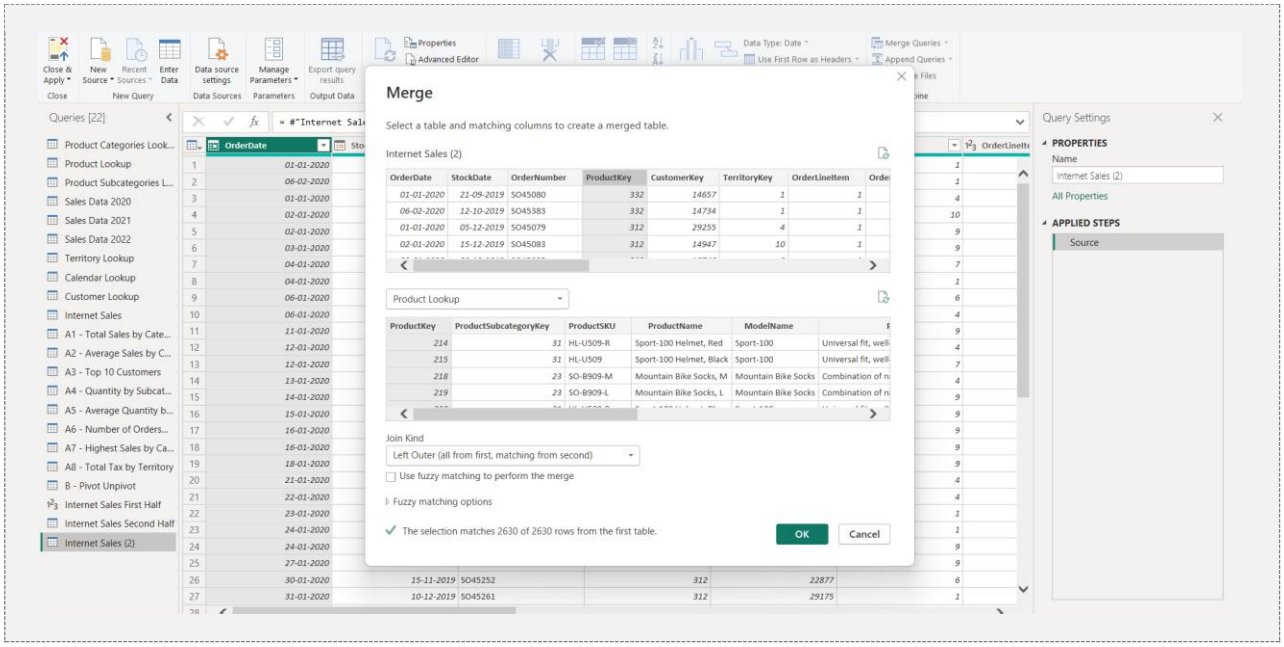
1. Merge FactInternetSales with DimProduct using ProductKey.

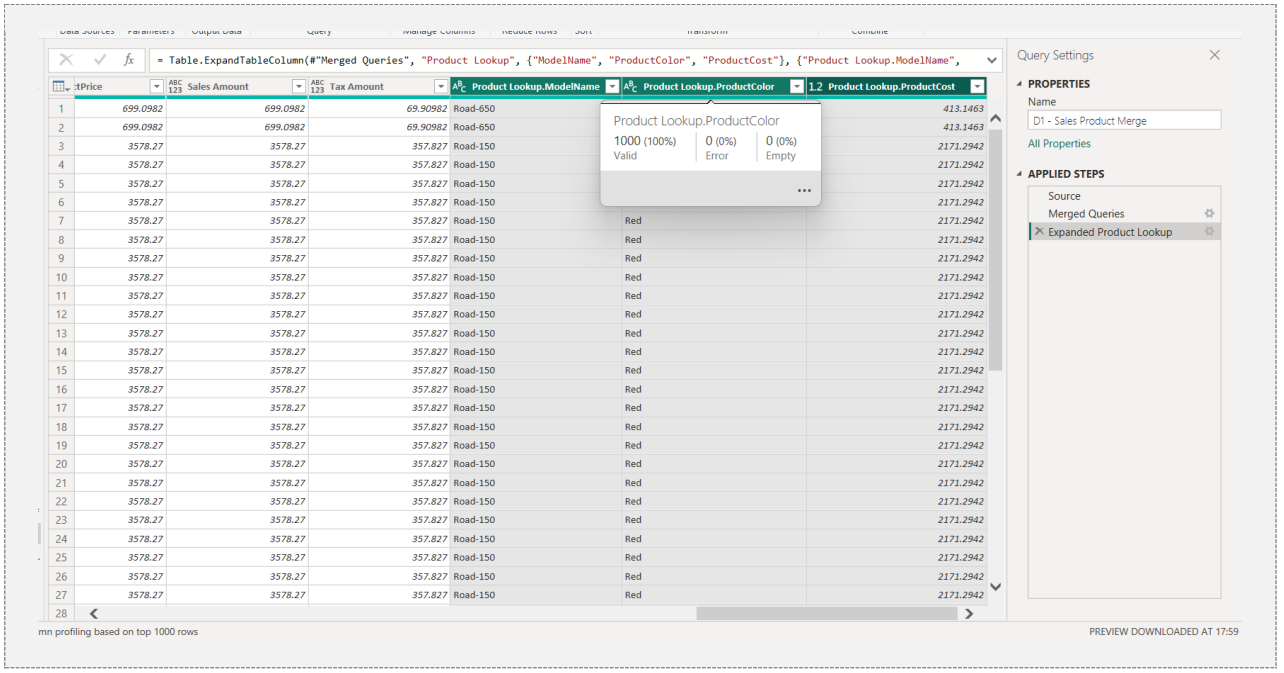
Merge is Power Query's join operation — it looks up matching rows from a second table based on a shared key, similar to a SQL JOIN or an Excel VLOOKUP, but returning a nested table column.

1. Select FactInternetSales. 2. Home > Merge Queries. 3. Choose DimProduct and match on ProductKey, join kind = Left Outer.

2. Expand the merged table to include English Product Name, Color and Standard Cost.

The merge step only attaches a nested table column Expand is what actually pulls specific fields out of that nested table into flat, usable columns. 1. Click the expand icon on the new merged column. 2. Tick only English Product Name, Color and Standard Cost. 3. Uncheck 'Use original column name as prefix' for cleaner column names.





3. Merge the result with DimProductSubcategory.

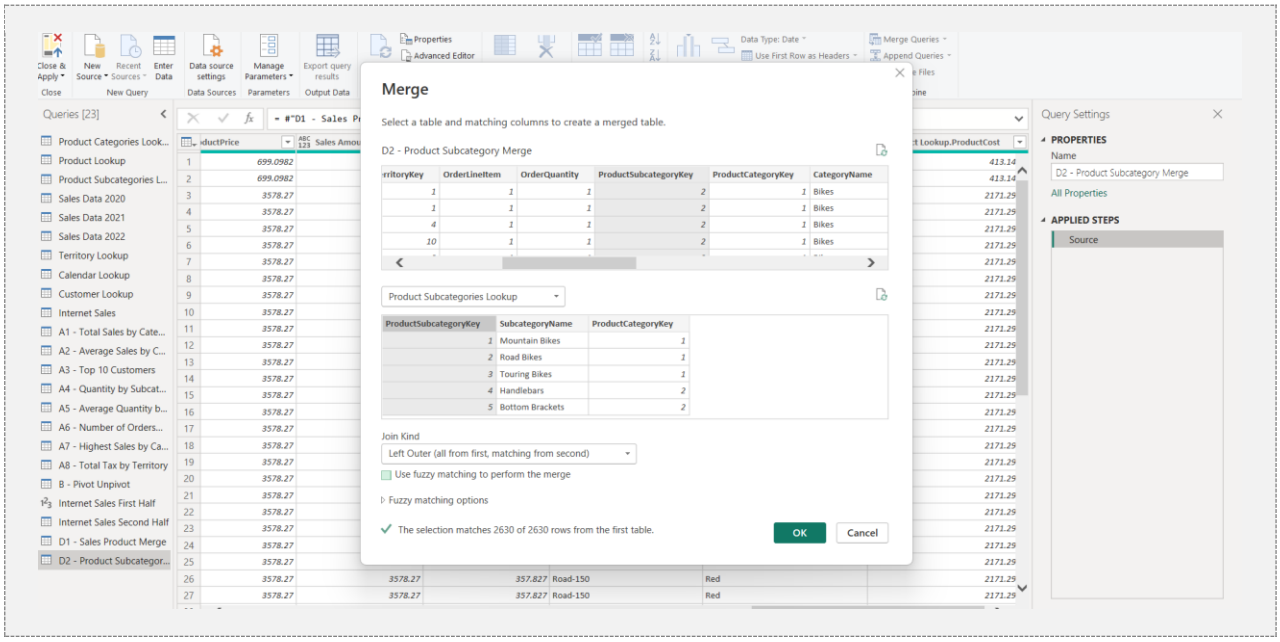
The same Merge pattern is repeated to walk up the product hierarchy — this time joining on ProductSubcategoryKey to bring in subcategory details.

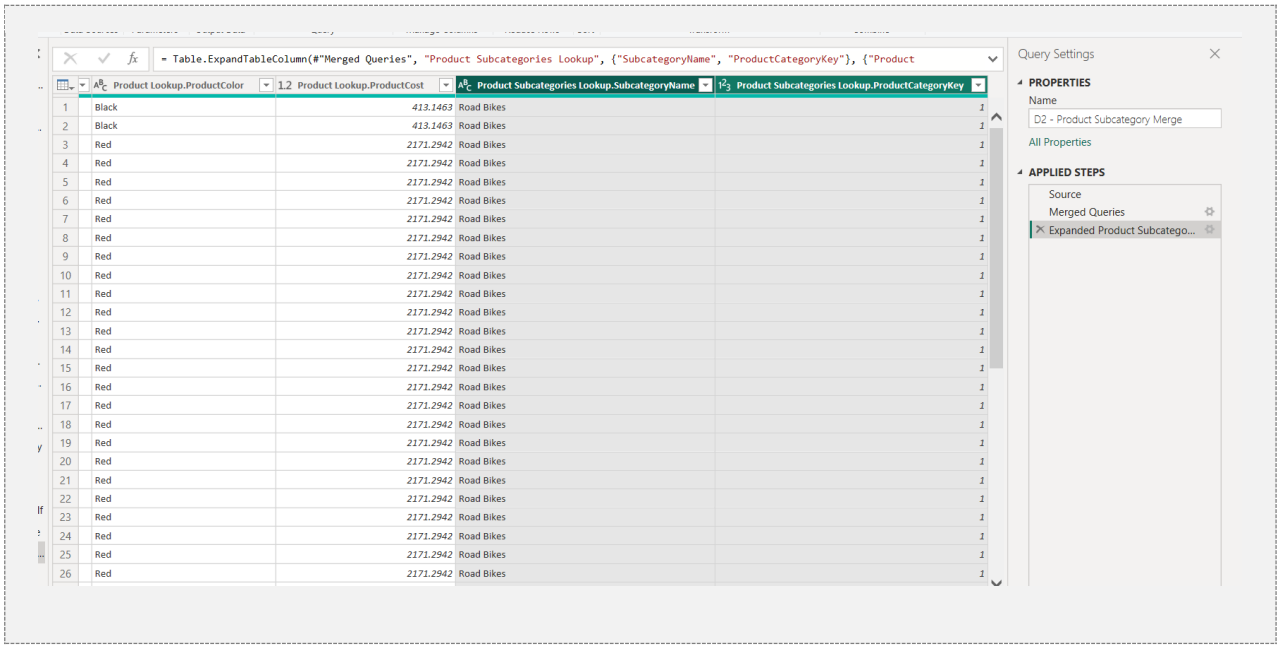
1. Select the table from D2.
2. Home > Merge Queries, choose DimProductSubcategory.
3. Match on ProductSubcategoryKey, join kind = Left Outer.

4. Expand English Product Subcategory Name.

As with D2, only the specific field needed is pulled out of the nested table rather than every column from DimProductSubcategory.

1. Click the expand icon on the merged column.
2. Tick only English Product Subcategory Name.
3. Uncheck the prefix option.



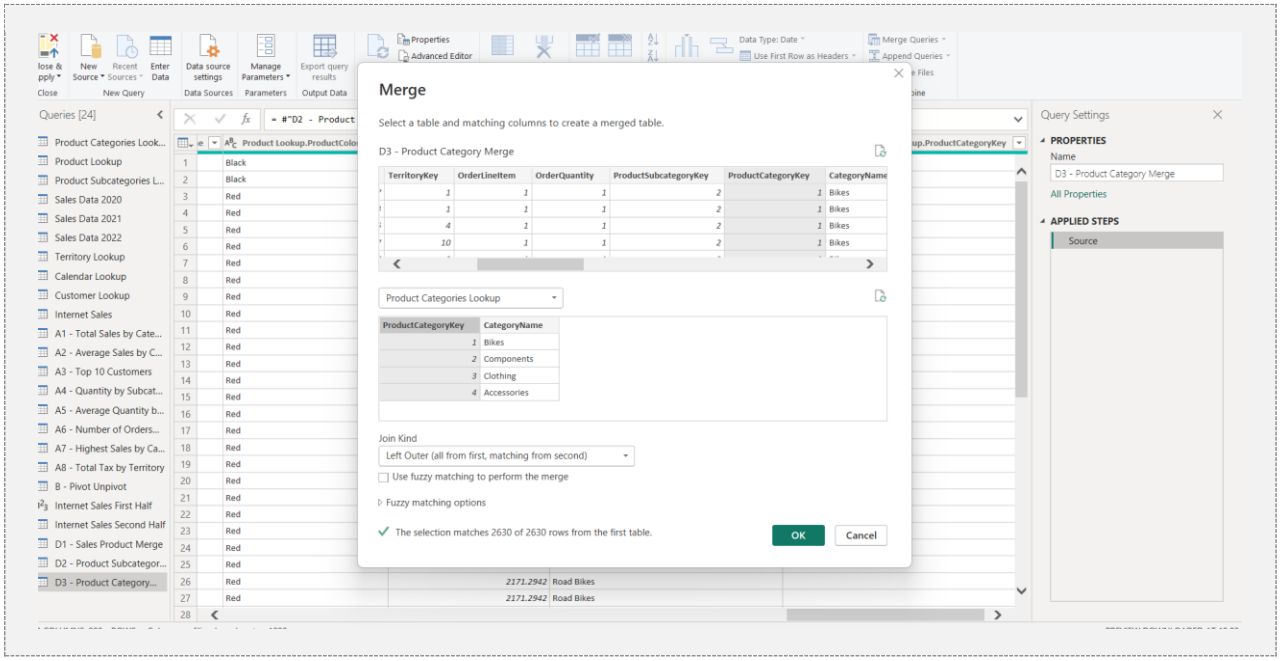


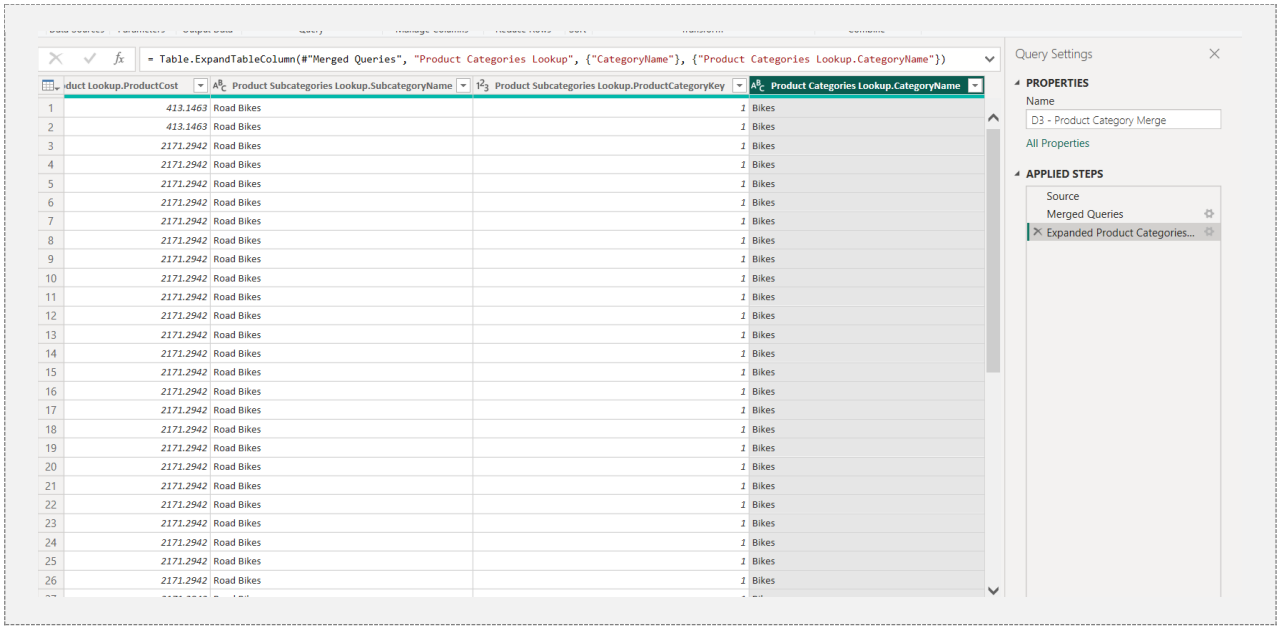
5. Merge the result with DimProductCategory.
One more hop up the hierarchy . Subcategory links to Category via ProductCategoryKey, completing the Product to Subcategory to Category chain. 1. Select the table from D4. 2. Home > Merge Queries, choose DimProductCategory. 3. Match on ProductCategoryKey, join kind = Left Outer.

6. Expand English Product Category Name.
The final expand step surfaces the top-level category label (Bikes / Components / Clothing / Accessories), completing the flat, analysis-ready table. 1. Click the expand icon on the merged column. 2. Tick only English Product Category Name. 3. Uncheck the prefix option.

7. Verify that every sales record contains the corresponding product category information.
A Left Outer join keeps every row from the sales table even if no match is found in the lookup — checking for blank Category values afterwards catches any ProductKey that didn't match anything.

1. Scroll through the final merged table for blank Category values. 2. Or add a quick Group By / Count of Rows filtered where Category is blank as a formal check. 3. Confirm the blank count is zero.



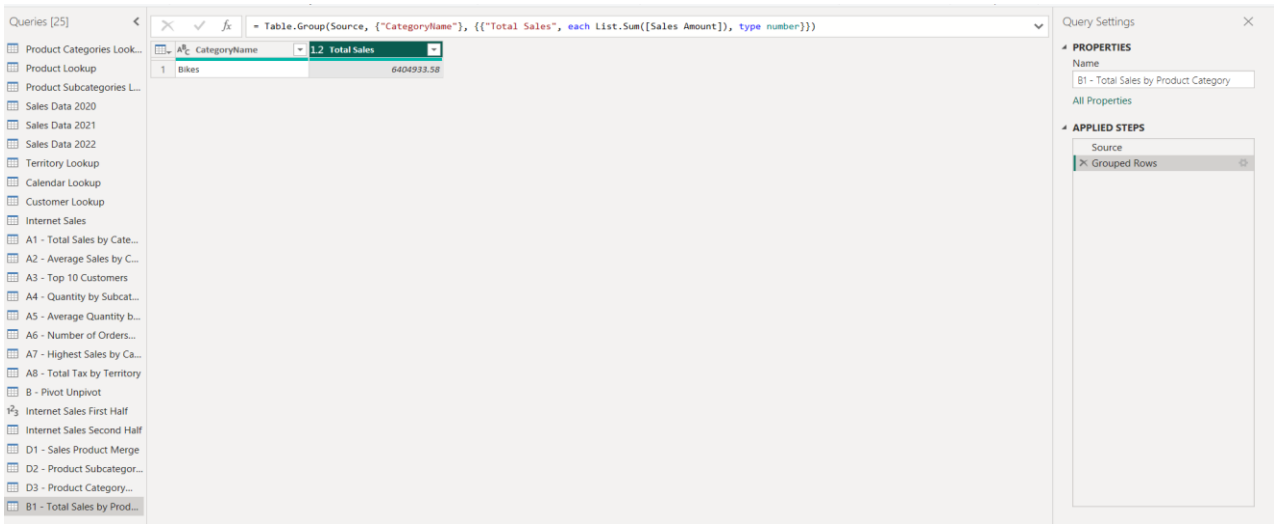


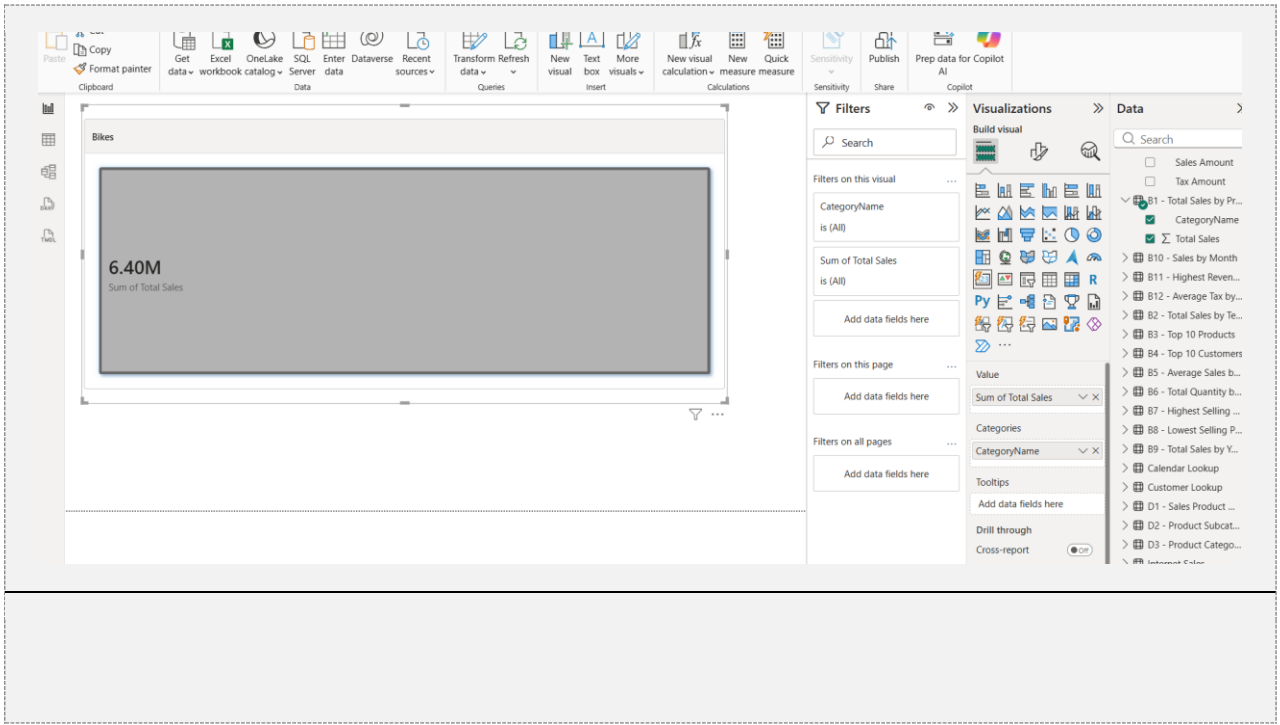
Part B: Sales Analysis

With a clean, fully merged working table in place, Part B layers DAX measures on top to answer specific business questions — total and average sales, top/bottom performers, trends over time, and territory/tax comparisons.

1. Calculate Total Sales by Product Category.
This is the headline revenue-by-category measure most dashboards lead with — a simple SUM of Sales Amount sliced by category.

1. Create a DAX measure: Total Sales = SUM(Sales Amount).
2. Place Product Category on the rows/axis of a Table or Bar chart.
3. Add Total Sales as the value.

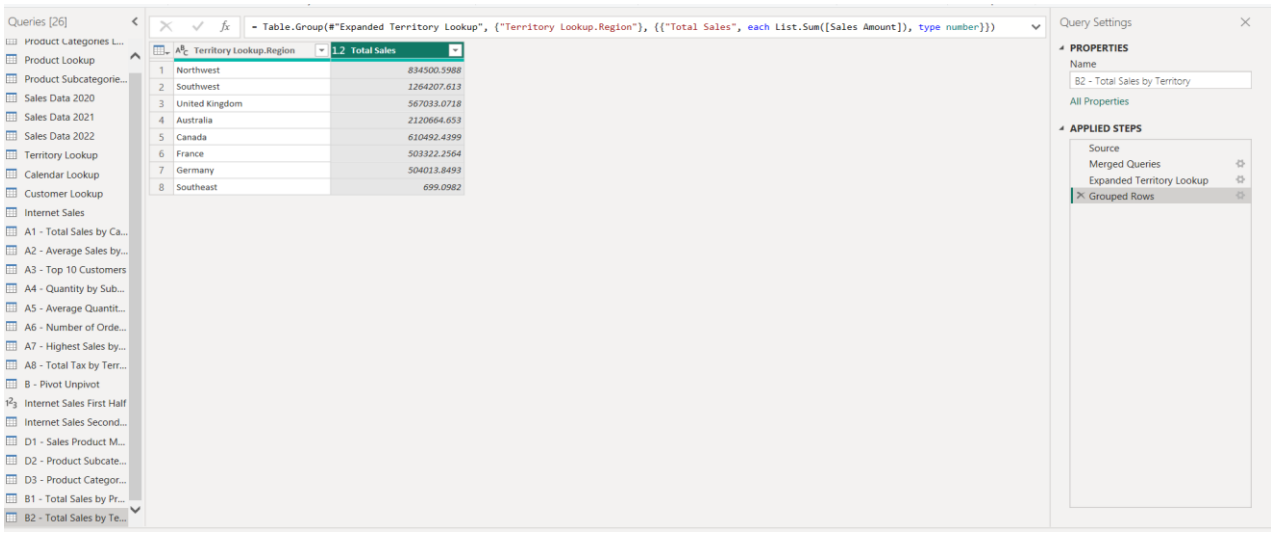


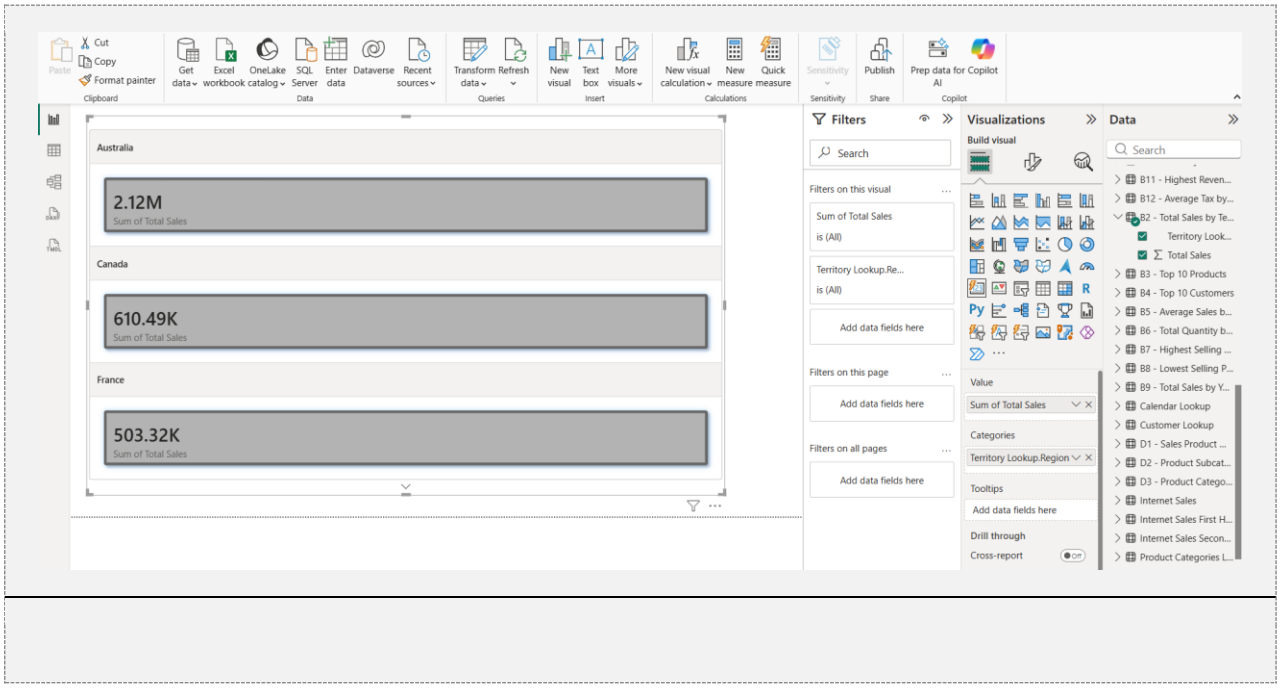


2. Calculate Total Sales by Sales Territory.

The same Total Sales measure sliced by Region instead of Category reveals which geographies are driving revenue.

1. Reuse the Total Sales measure from PB1.
2. Place Sales Territory (Region) on the axis of a Bar chart or Map visual.
3. Add Total Sales as the value.

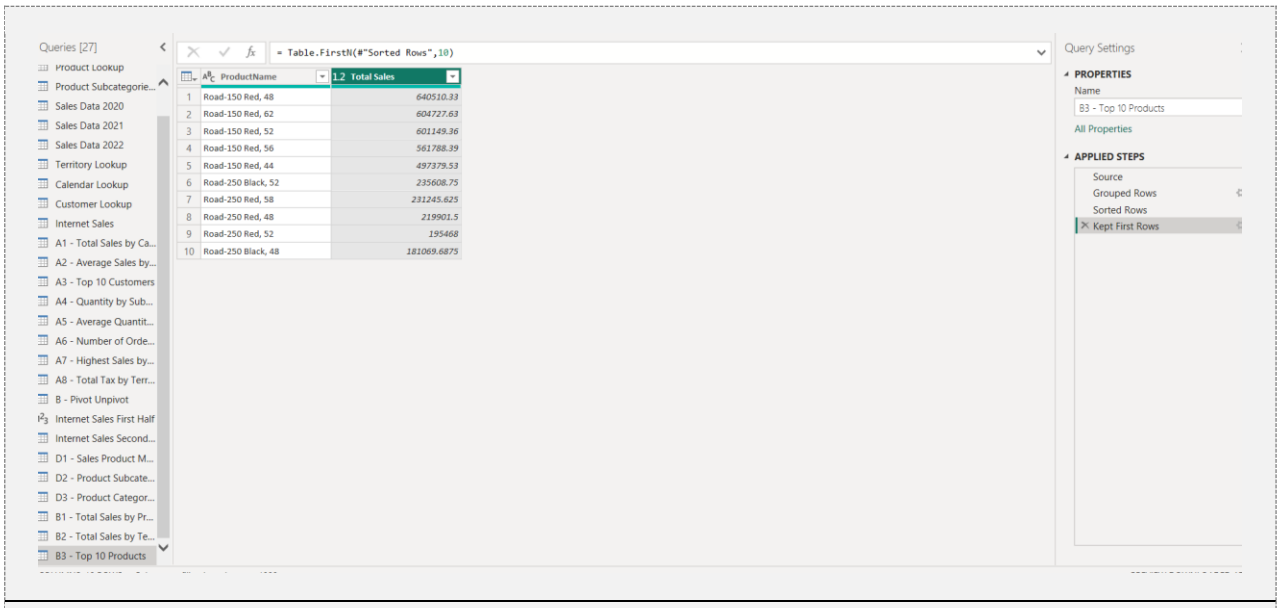


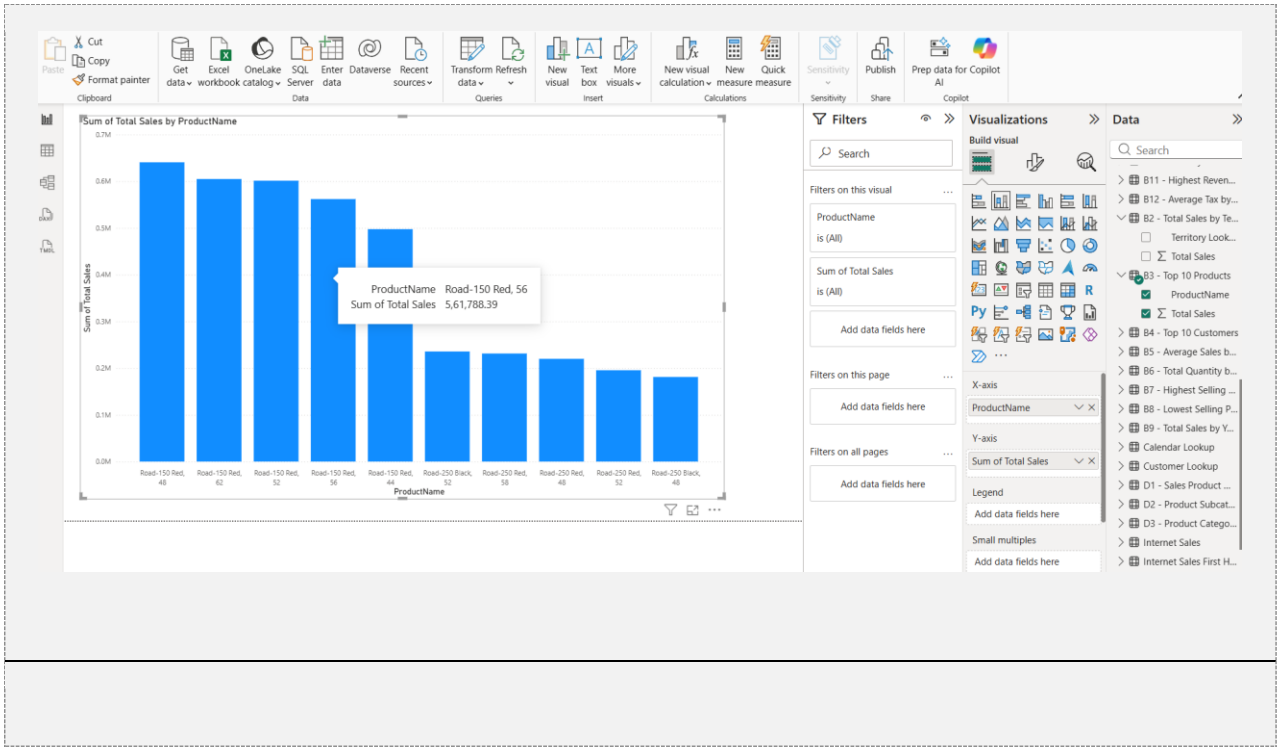


3. Find the Top 10 Products based on Sales Amount.

A Top N filter applied directly on a visual is often simpler than filtering the underlying data — it dynamically re-ranks if the data changes.

1. Add a Table/Bar chart with Product Name and Total Sales.
2. Open the visual-level filter pane on Product Name.
3. Set Filter Type = Top N, N = 10, By value = Total Sales.

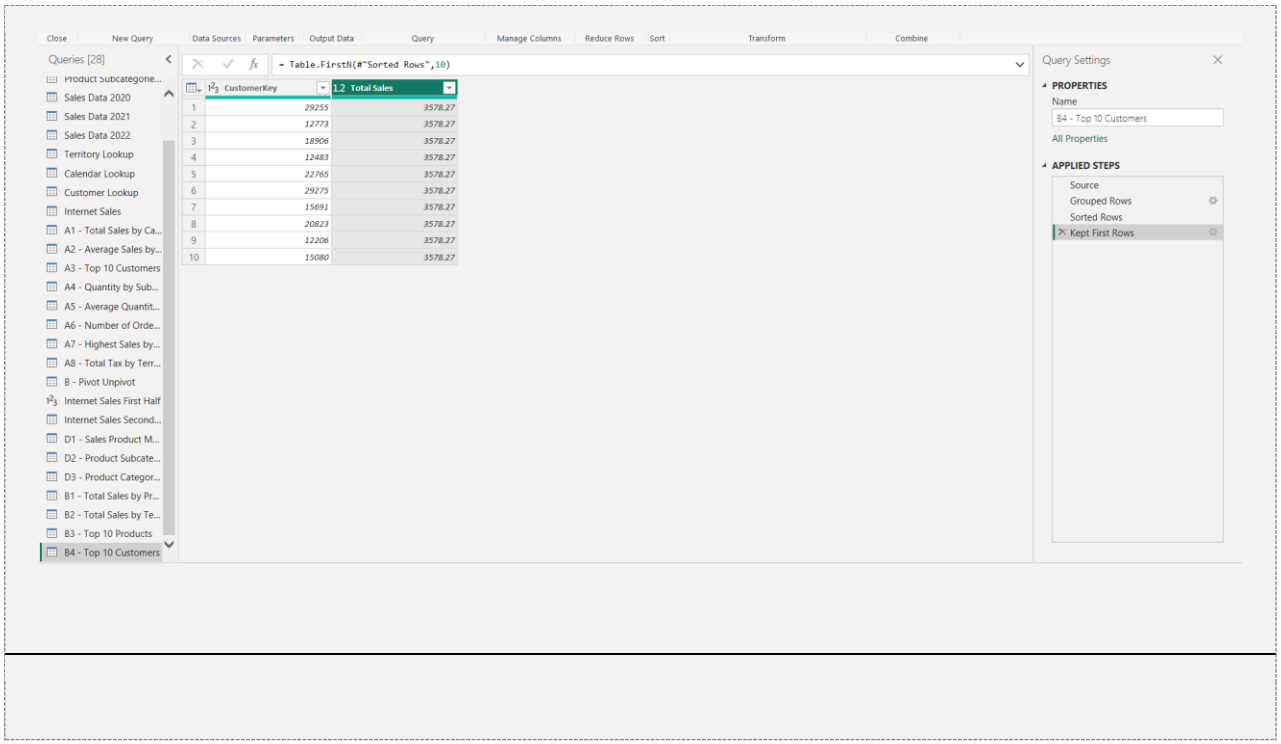


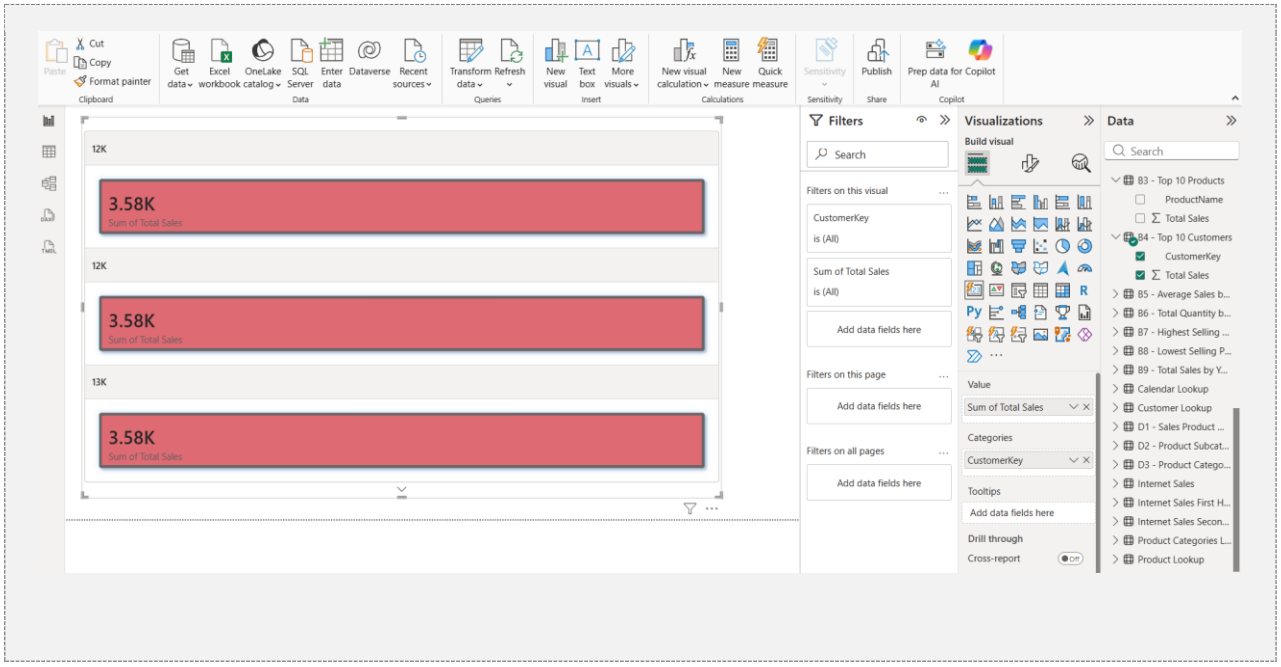


4. Determine the Top 10 Customers based on Sales Amount.

The same Top N filter pattern from PB3 applies equally to any dimension — here Customer instead of Product.

1. Add a Table/Bar chart with Customer Name and Total Sales.
2. Apply a Top N visual filter, N = 10, By value = Total Sales.

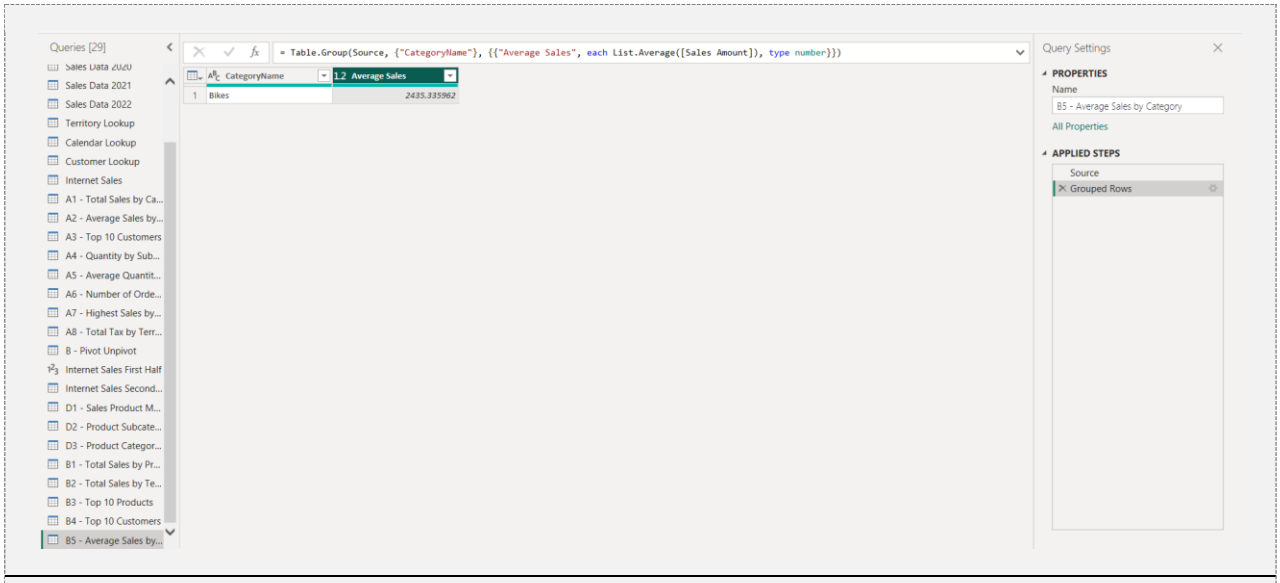


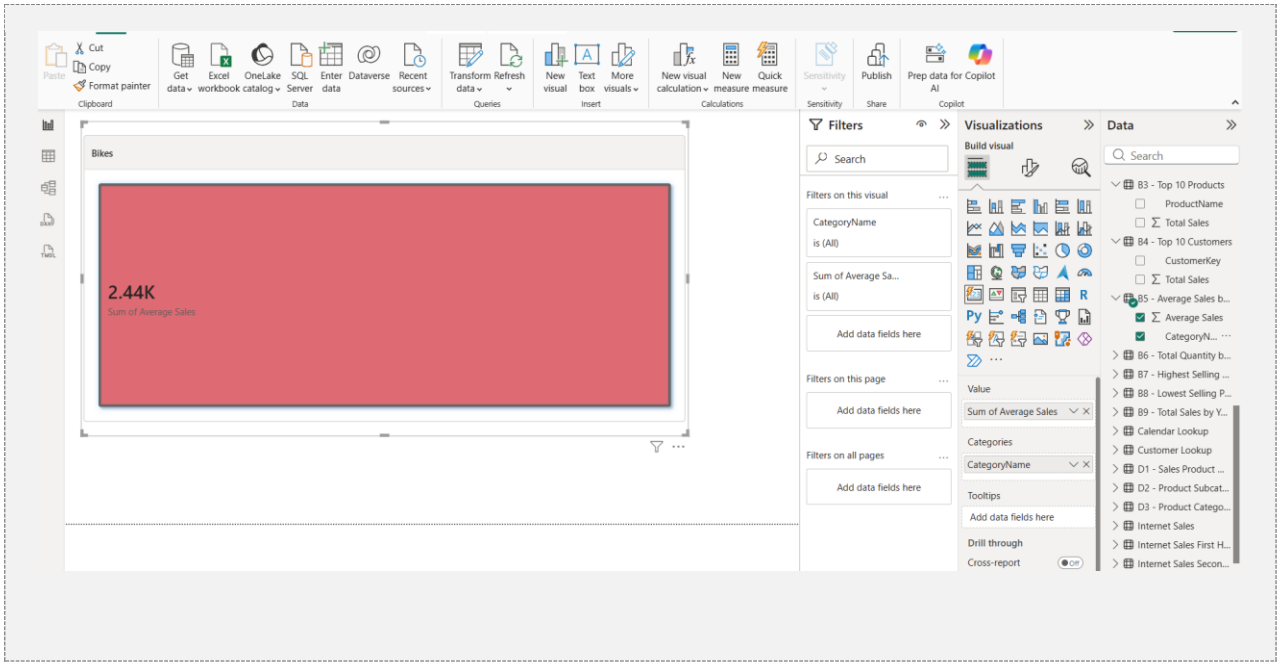


5. Calculate the Average Sales Amount by Product Category.

Average Sales per category is a very different signal from Total Sales — it shows typical transaction size rather than overall volume, useful for spotting categories with fewer but pricier items.

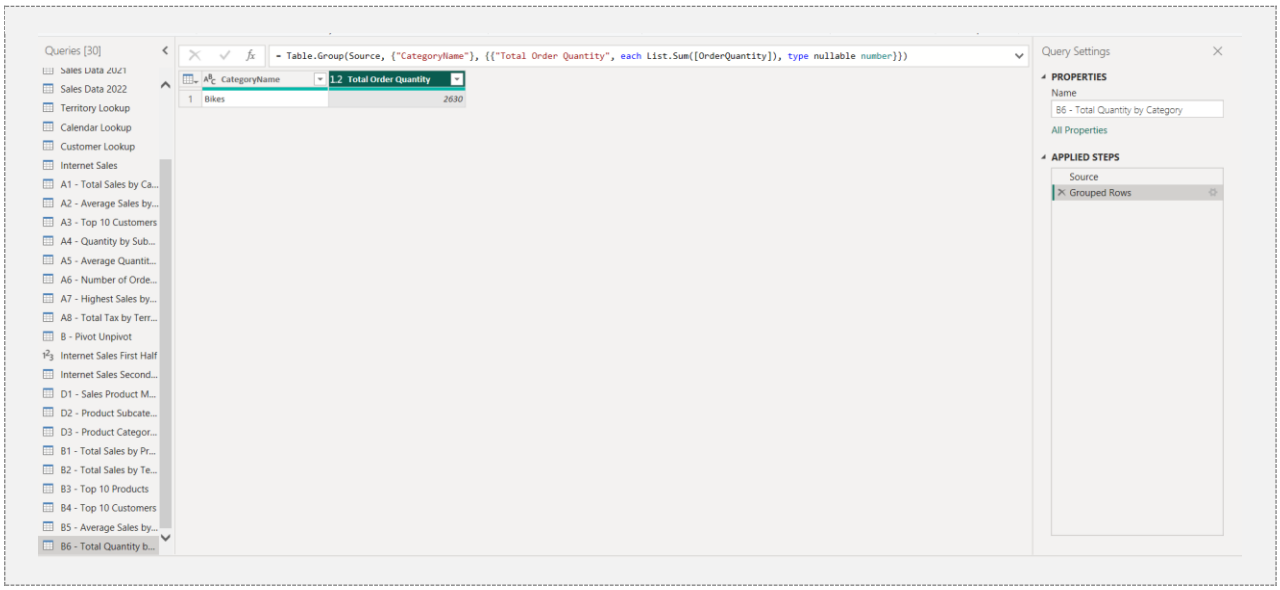
1. Create a DAX measure: Average Sales = AVERAGE(Sales Amount).
2. Place Product Category on rows of a Table/Bar chart.
3. Add Average Sales as the value.

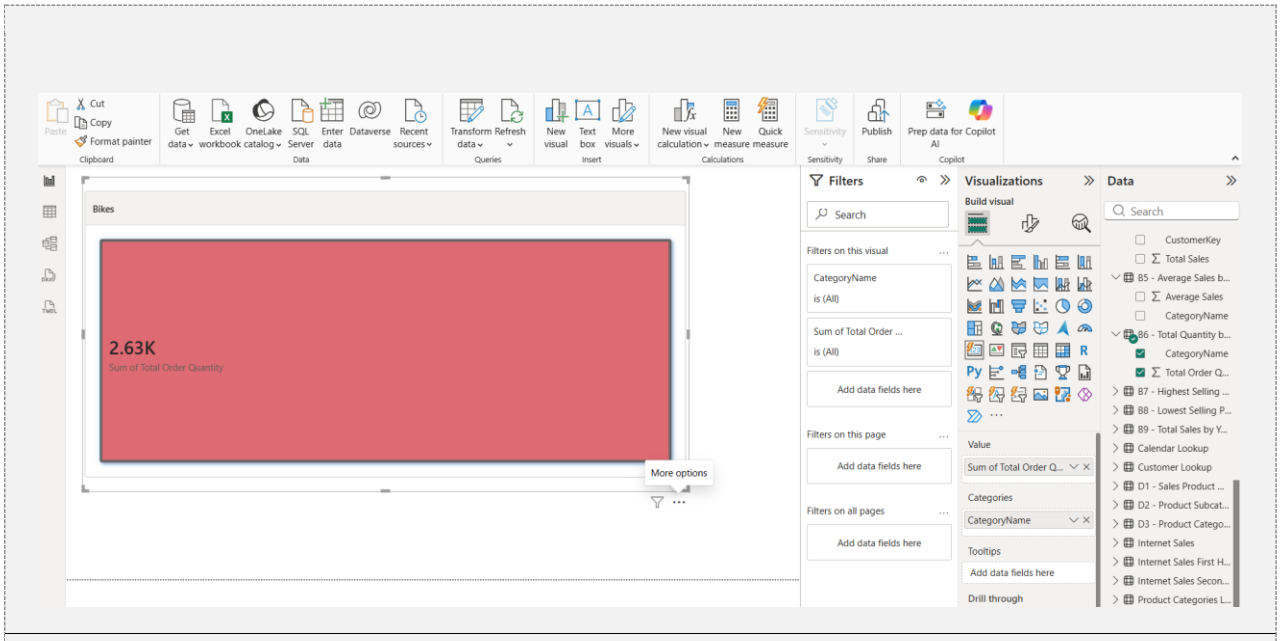




6. Find the Total Order Quantity for each Product Category.
Pairing this with PB1 gives a fuller picture — a category can have low Total Sales but high Order Quantity if its items are cheap and high-volume.

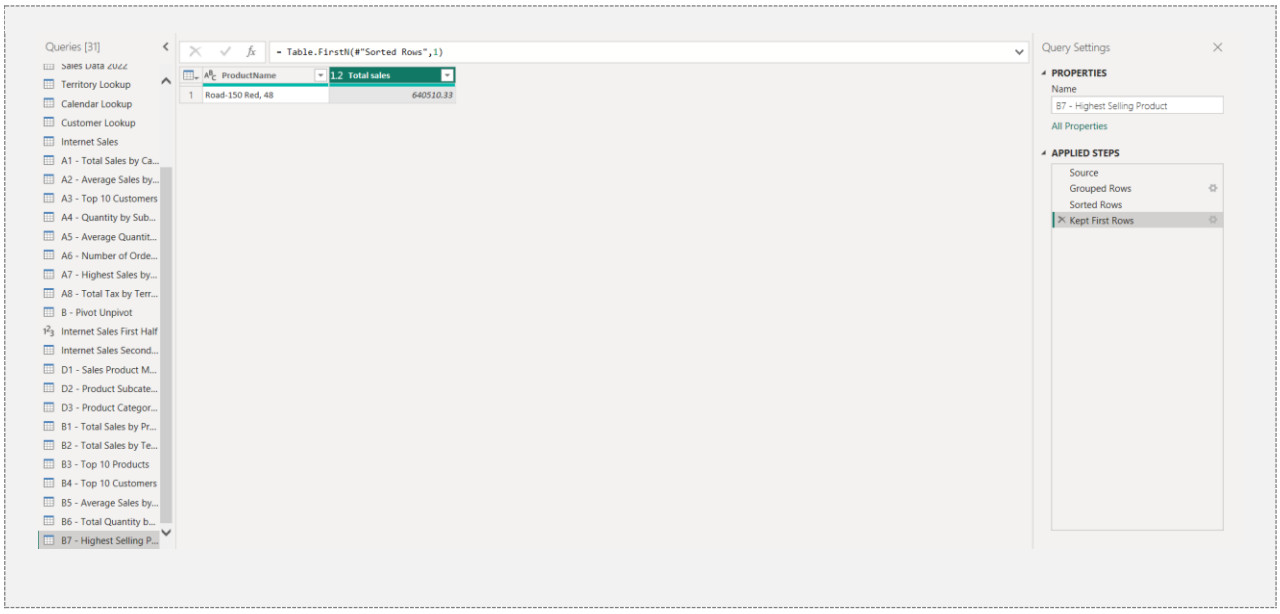
1. Create a DAX measure: Total Order Quantity = SUM(Order Quantity).
2. Place Product Category on rows with Total Order Quantity as the value.

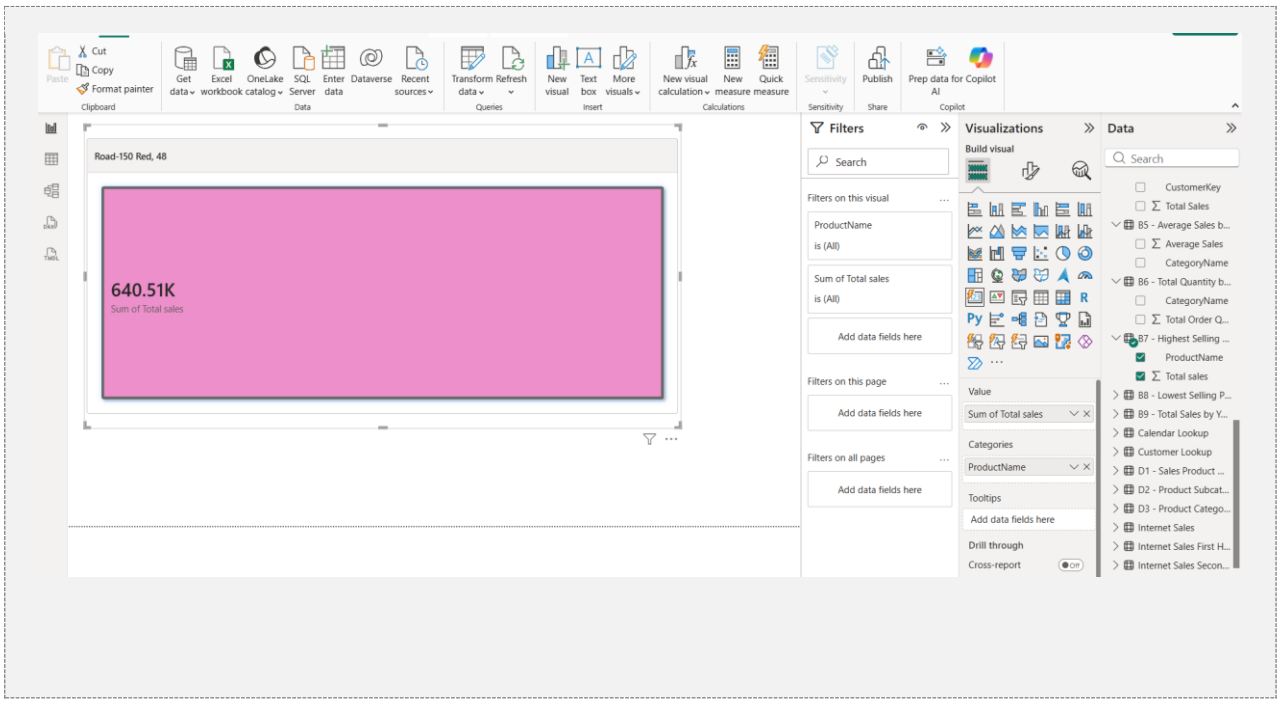




7. Determine the highest-selling product.
Rather than eyeballing a sorted list, a dedicated single-value Card makes the top performer instantly visible on the dashboard.

1. Sort the Product/Total Sales table from PB3 descending.
2. Alternatively, write a TOPN DAX measure and bind it to a Card visual.

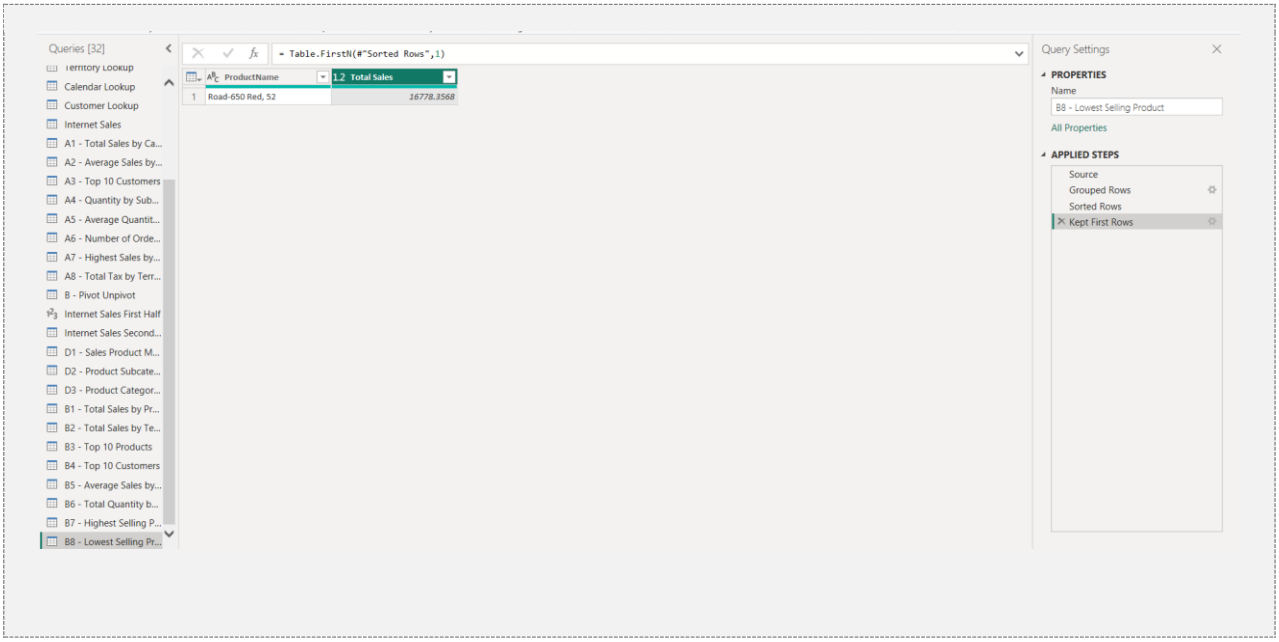


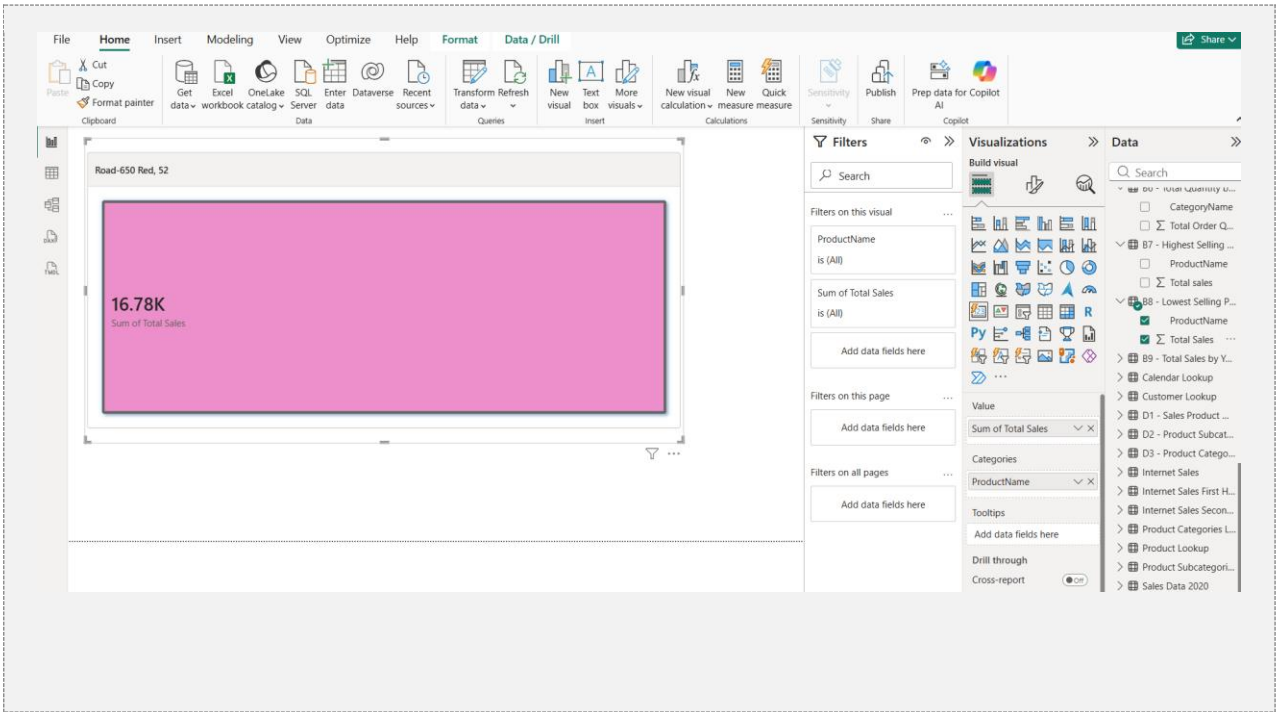


8. Identify the lowest-selling product.

The mirror image of PB7 — useful for spotting underperforming SKUs that might be candidates for discontinuation or a promotion.

1. Sort the Product/Total Sales table ascending (excluding zero/blank sales).
2. Alternatively, write a BOTTOMN DAX measure and bind it to a Card visual.

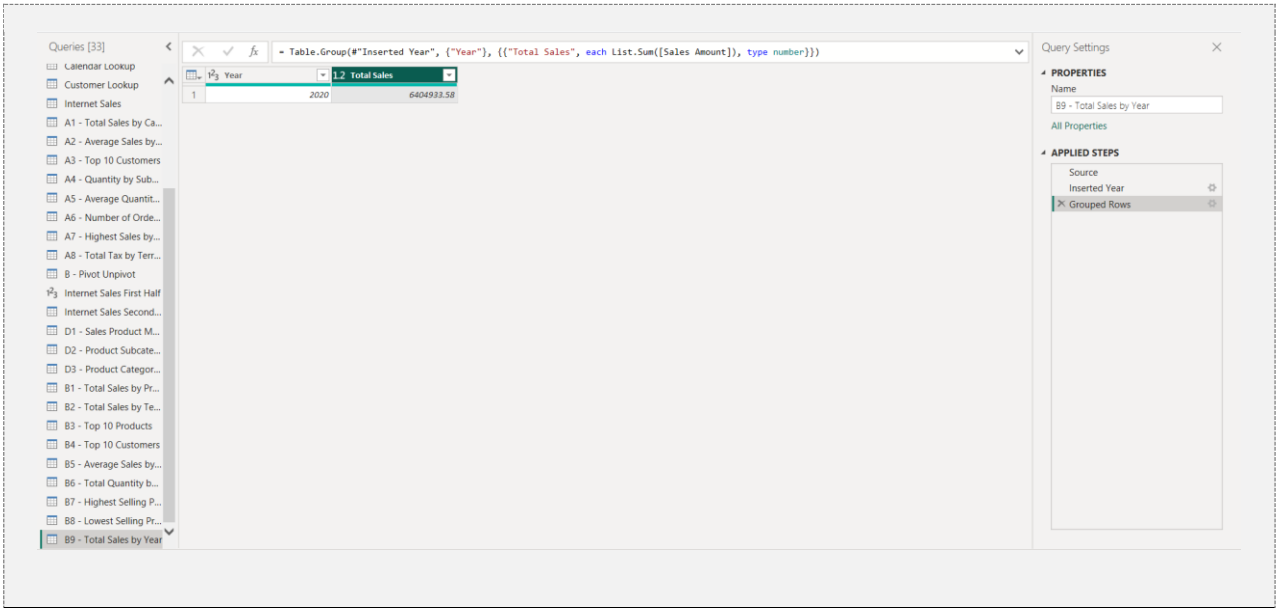


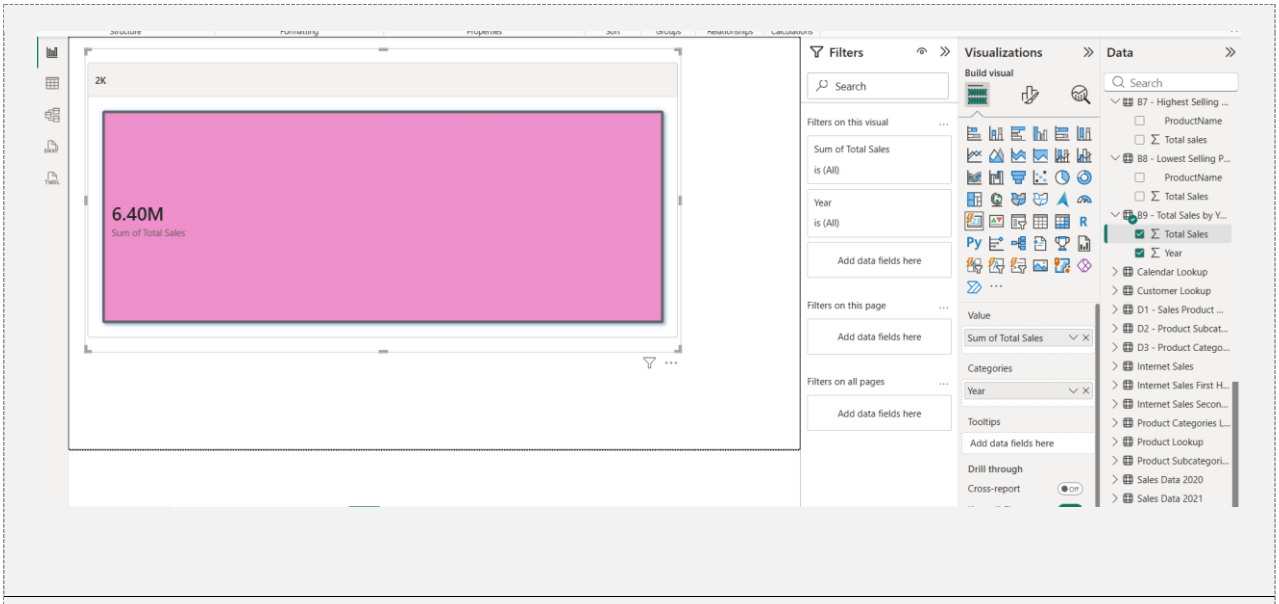


9. Calculate the total sales for each calendar year.

A year-over-year Total Sales view is the standard first chart in most sales dashboards — it immediately shows growth or decline trend.

1. Place Order Date (Year) on the axis of a Column chart.
2. Add Total Sales as the value.

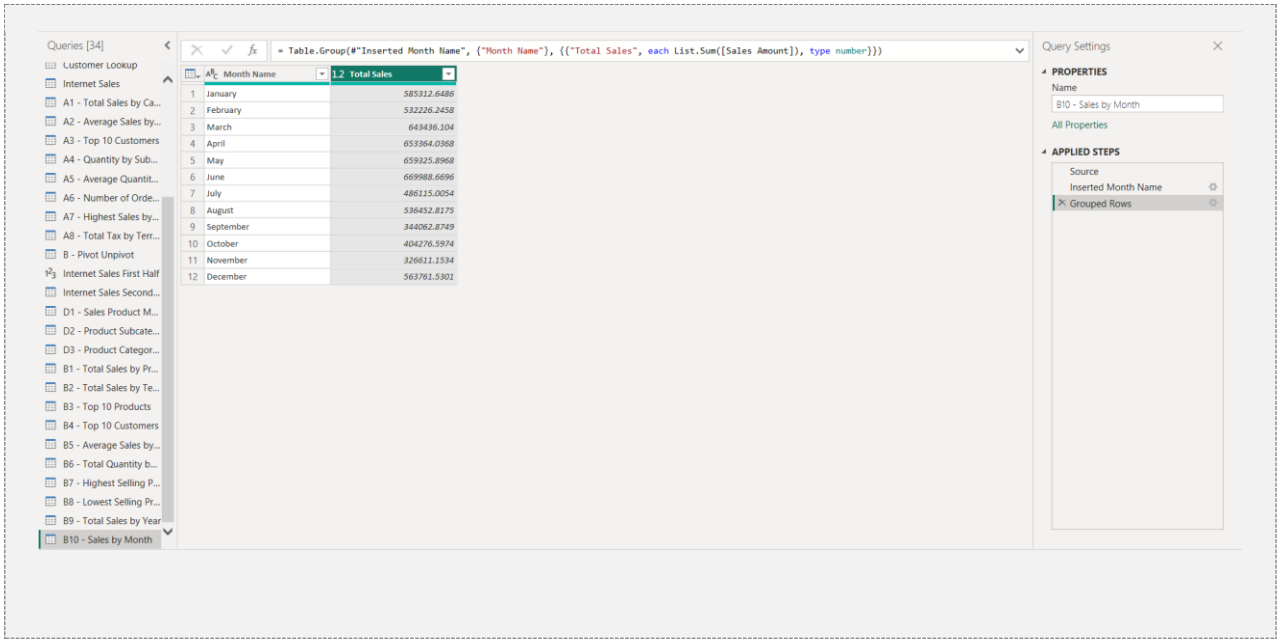


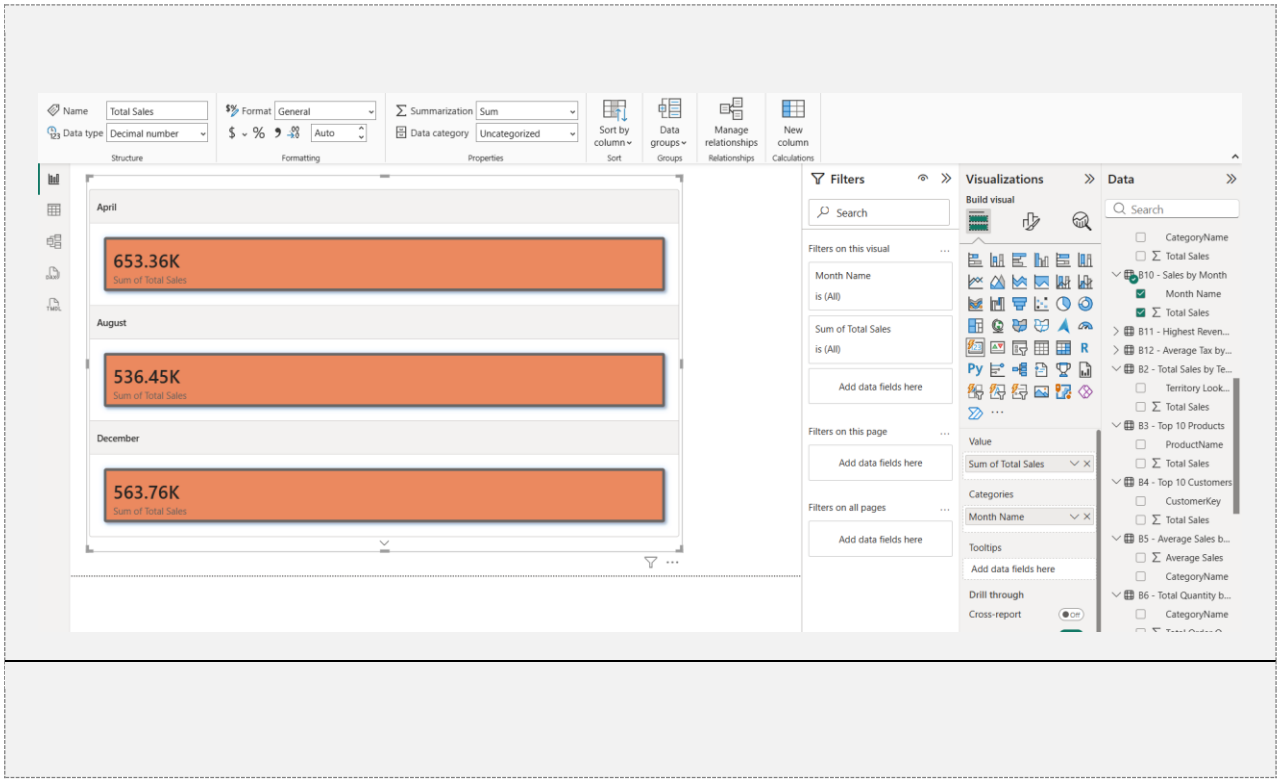


10. Compare sales across different months.

Month-level trend reveals seasonality that a yearly view hides — a sort column by Month Number keeps the axis in calendar order instead of alphabetical.

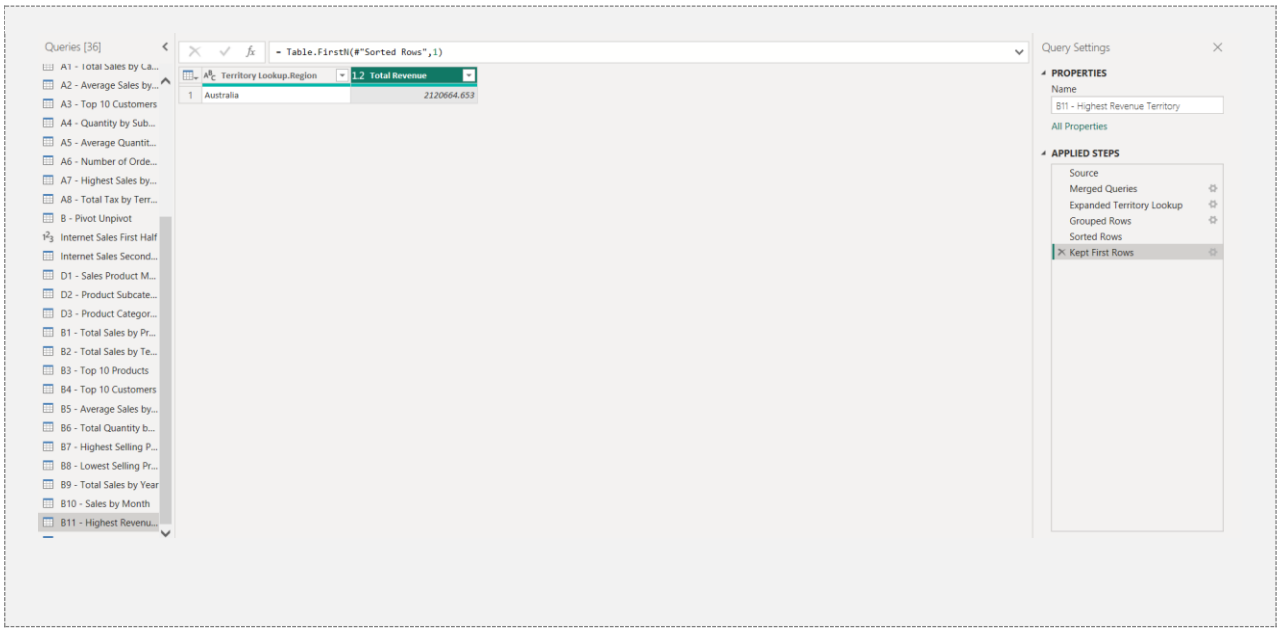
1. Extract Month Name and a numeric Month Number from Order Date.
2. Sort the Month Name column by Month Number (Column Tools > Sort by Column).
3. Place Month Name on a Line/Column chart with Total Sales as the value.

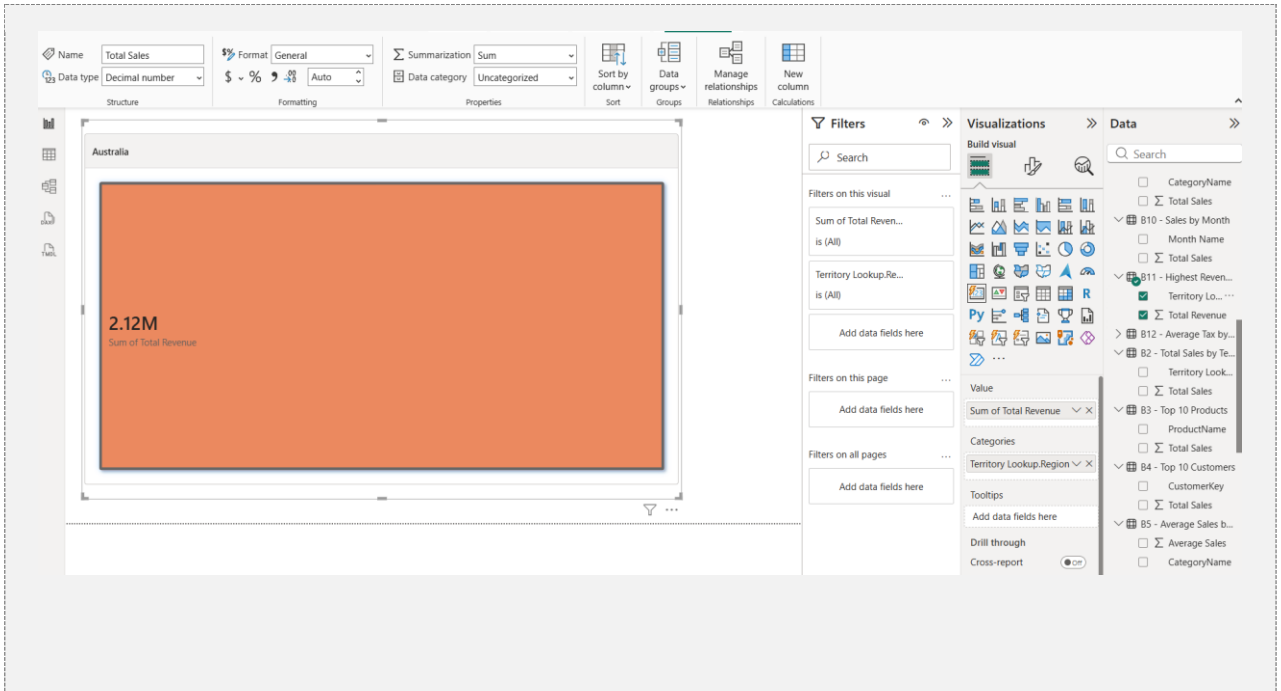




11. Determine which sales territory generated the highest revenue.
Same idea as PB7/PB8 but applied to Territory — a single Card is clearer for a dashboard headline than making the reader scan a bar chart.

1. Sort the Territory/Total Sales chart from PB2 descending.
2. Alternatively, use a MAXX DAX measure over Region totals bound to a Card.

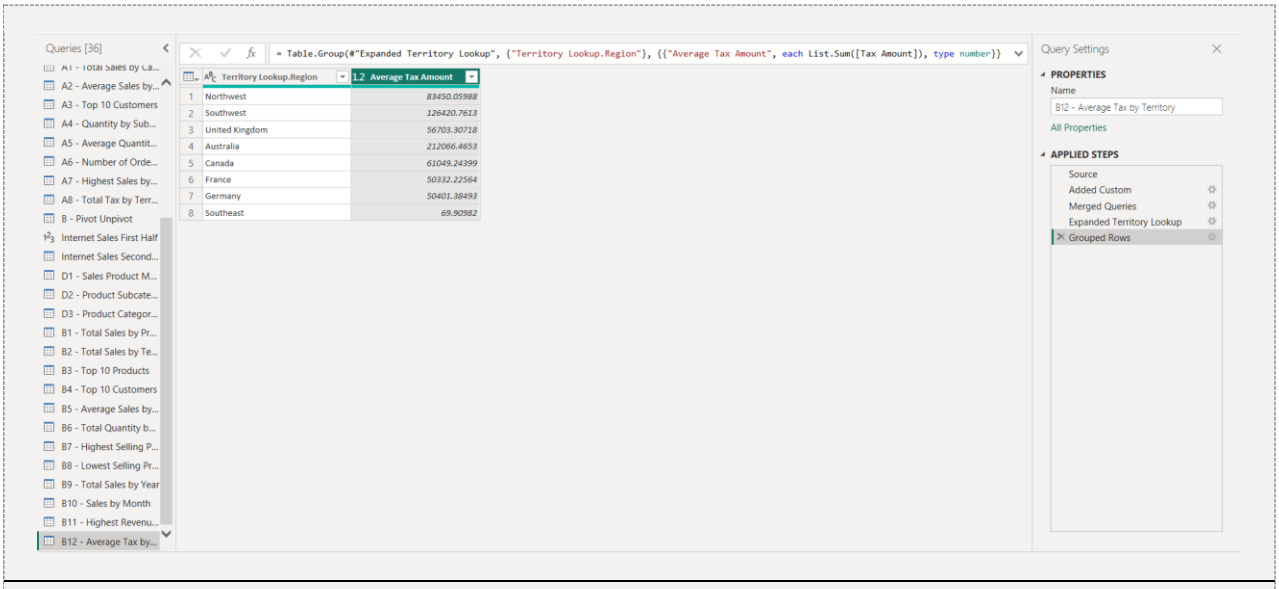


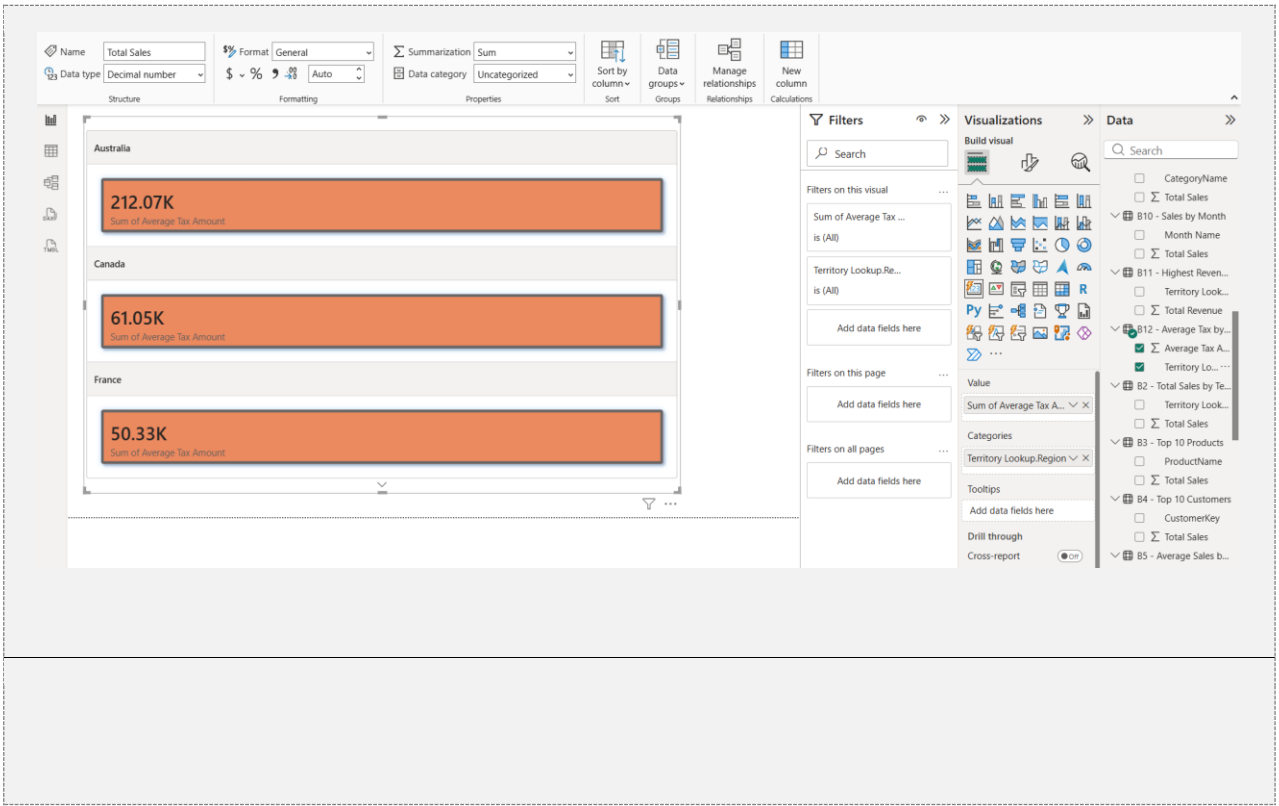


12. Find the average tax amount collected per territory.

Average Tax per territory (as opposed to total tax from A8) normalizes for order volume, showing which regions have higher-value transactions on average.

1. Create a DAX measure: Average Tax Amount = AVERAGE(Tax Amount).
2. Place Region on rows of a Table/Bar chart with Average Tax Amount as the value.



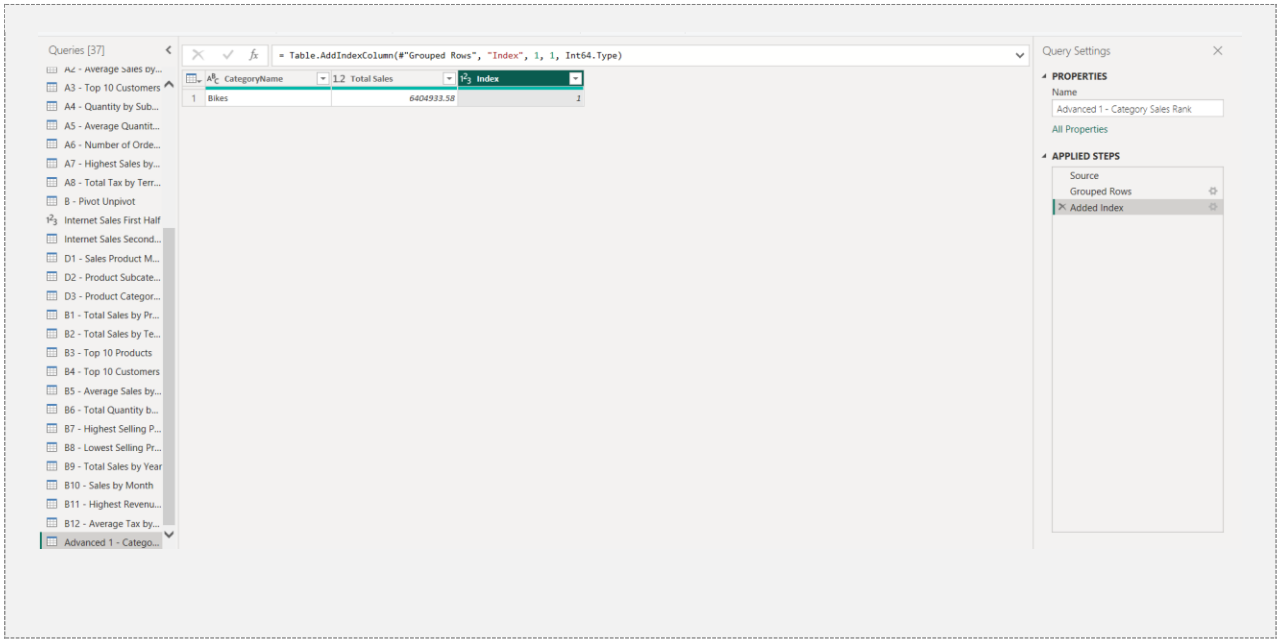


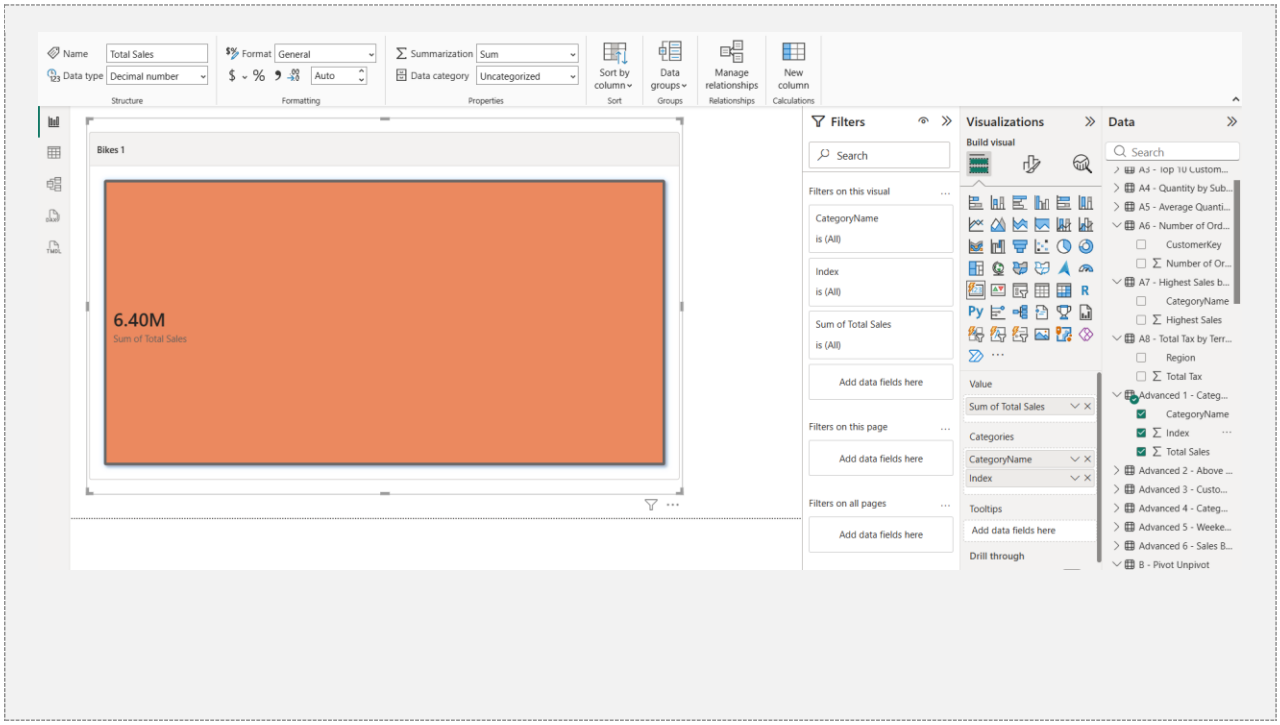
Advanced Tasks

The advanced tasks combine ranking DAX functions (RANKX), context-aware aggregation (ALL, AVERAGEX), and Conditional Columns to turn raw numbers into ranked, percentage and banded views — the kind of derived fields that make a dashboard feel finished rather than just descriptive.

1. Rank product categories based on total sales.
RANKX assigns each row a numeric rank based on a measure, evaluated within the current filter context — useful for turning a sorted list into an explicit '#1, #2, #3' label.

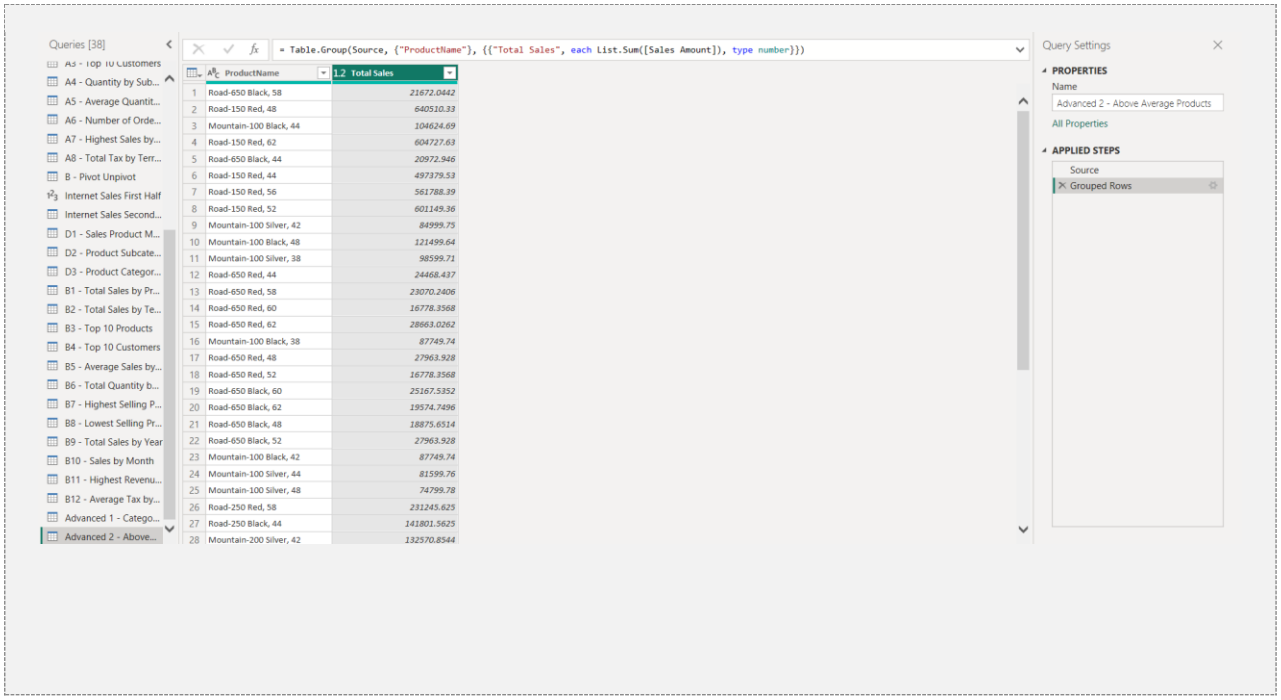
1. Write a DAX measure: Category Rank = RANKX(ALL(Category), [Total Sales]).
2. Place Category Rank alongside Category and Total Sales in a Table visual.
3. Sort by Category Rank.

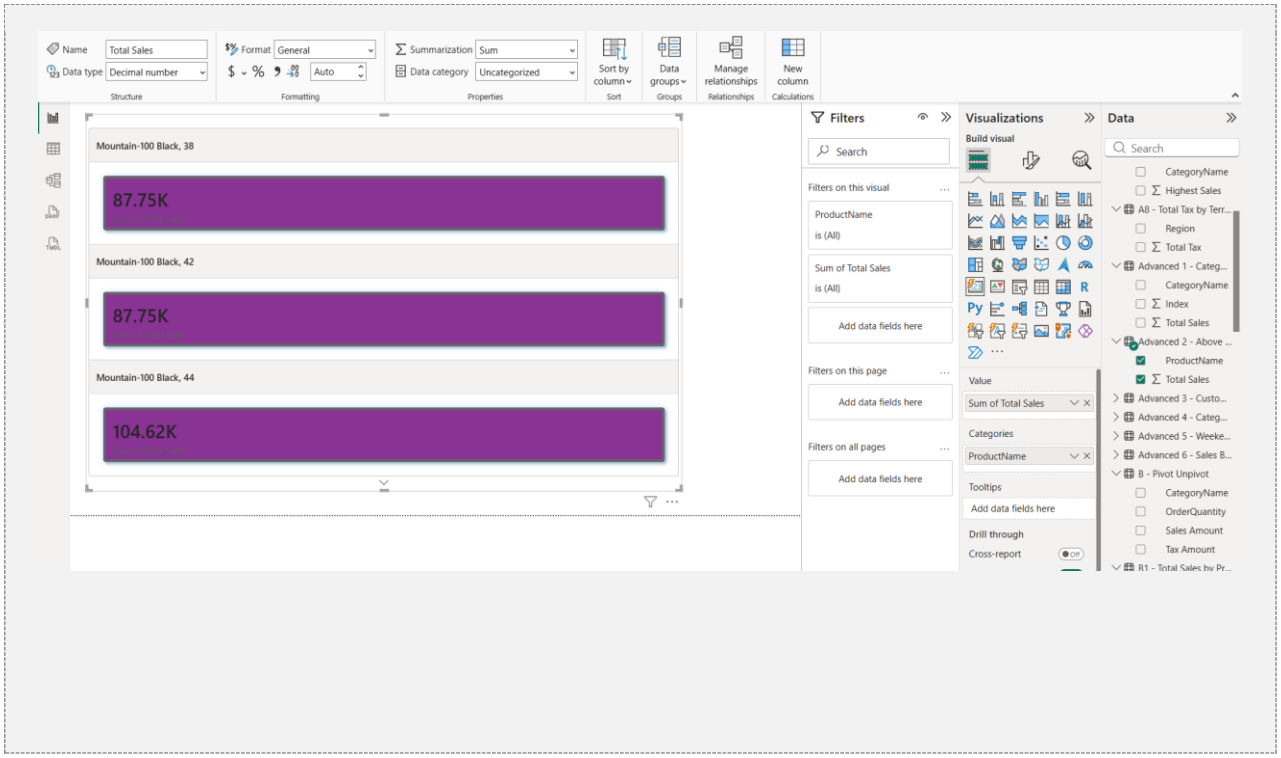




2. Identify products with above-average sales.
Comparing each product's total against the overall average (computed with ALL to ignore row-level filters) is a common way to flag over/under performers.

1. Write a measure for the overall average: Avg Product Sales = AVERAGEX(ALL(Product), [Total Sales]).
2. Add a calculated column/measure flag: IF([Total Sales] > [Avg Product Sales], "Above Average", "Below Average").
3. Filter or count products flagged Above Average.

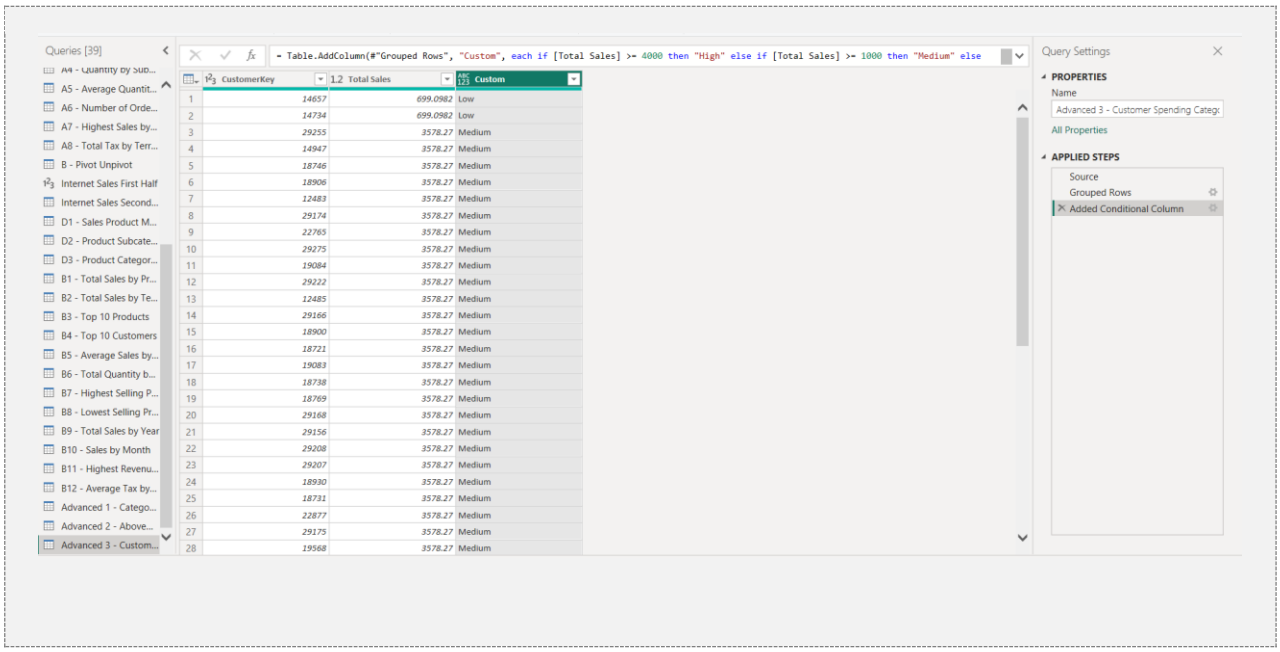


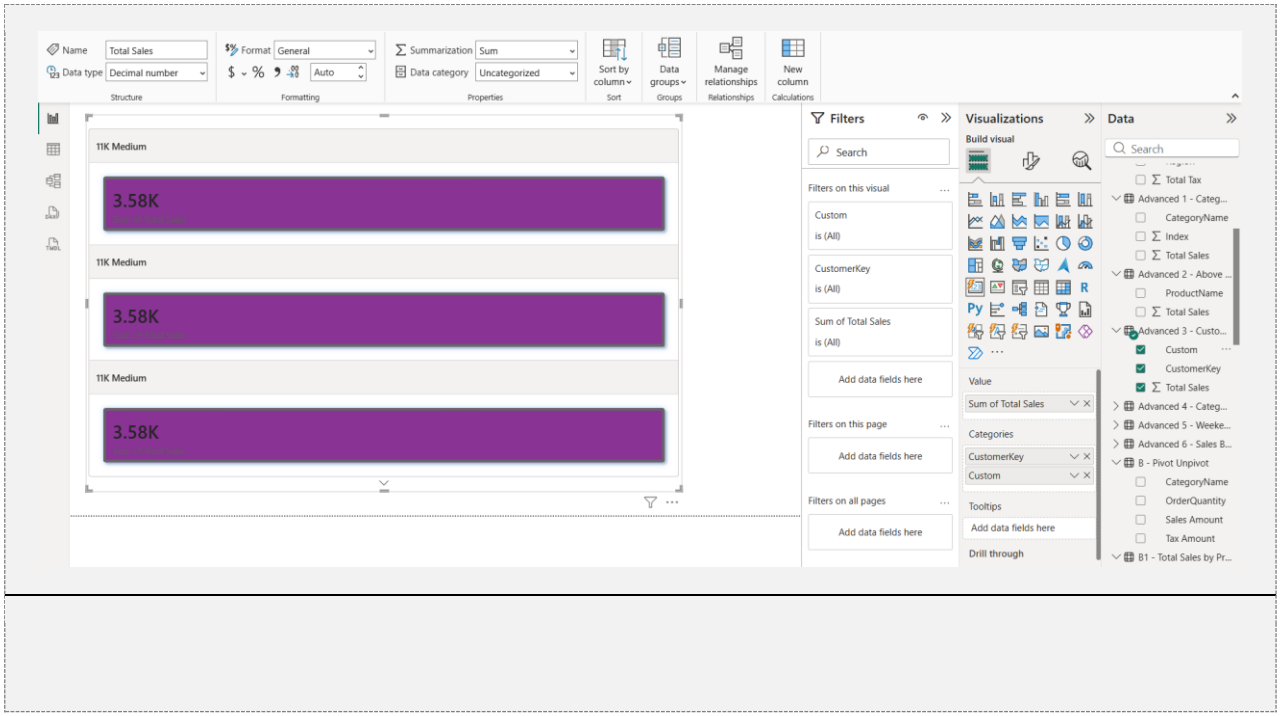


3. Group customers into High, Medium, and Low spending categories using Conditional Columns.

A Conditional Column buckets a continuous value (total customer spend) into discrete labels, which is much easier to slice, filter and visualize than raw currency values.

1. Group Sales Analysis by CustomerKey, Sum of Sales Amount, to get Total Customer Sales.
2. Add Column > Conditional Column on Total Customer Sales.
3. Set thresholds: Low if <= Rs 93.60, Medium if <= Rs 1,734.63, else High (tercile cutoffs of the customer base).

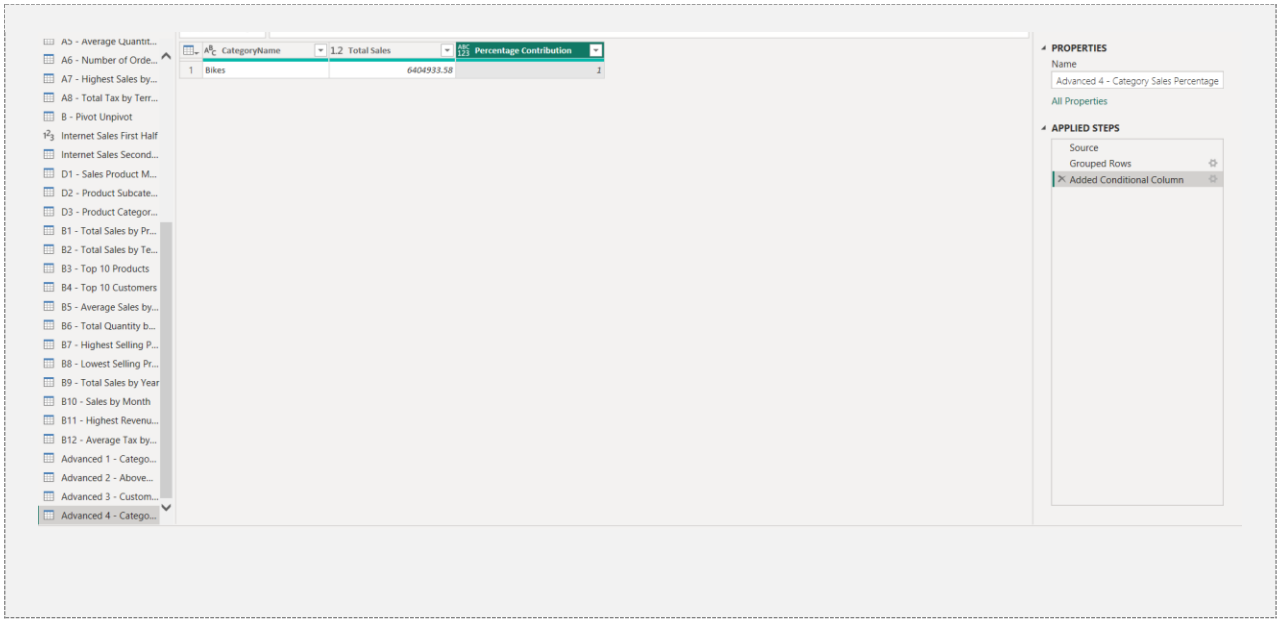


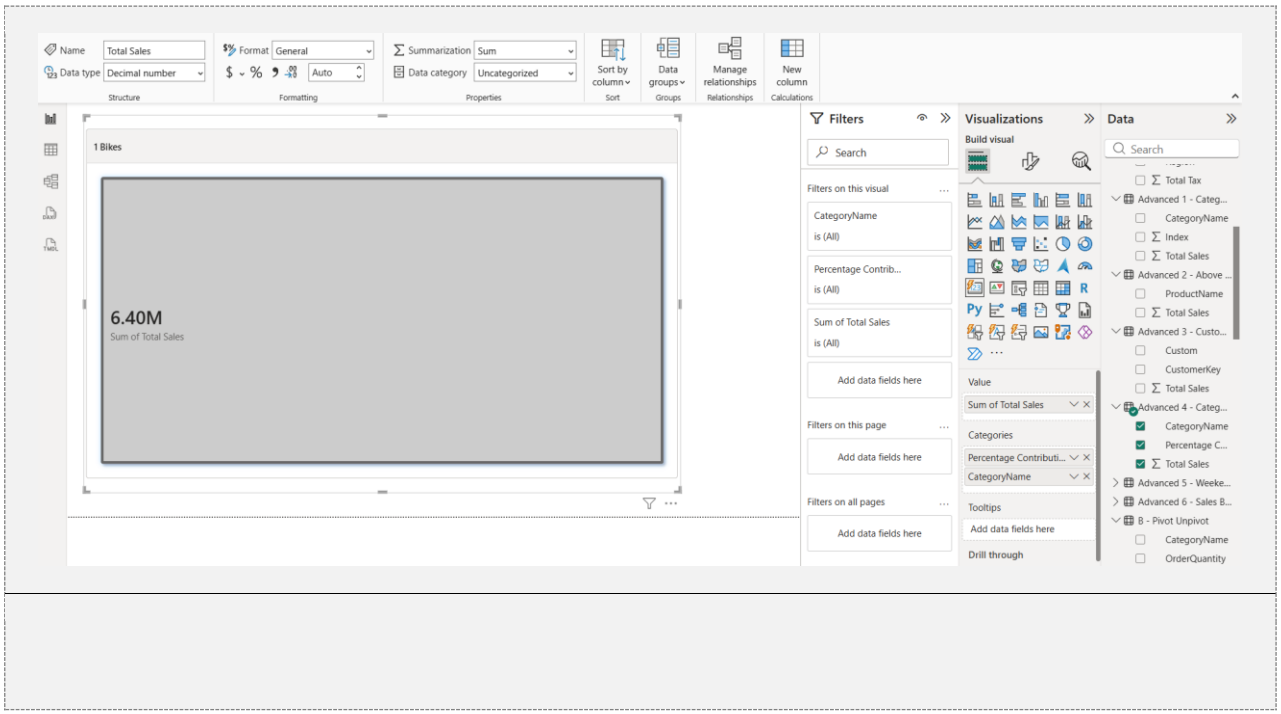


4. Calculate the percentage contribution of each product category to total sales.

A % of Total measure divides a category's sales by the grand total (again using ALL to remove the category filter for the denominator), giving a share-of-pie view.

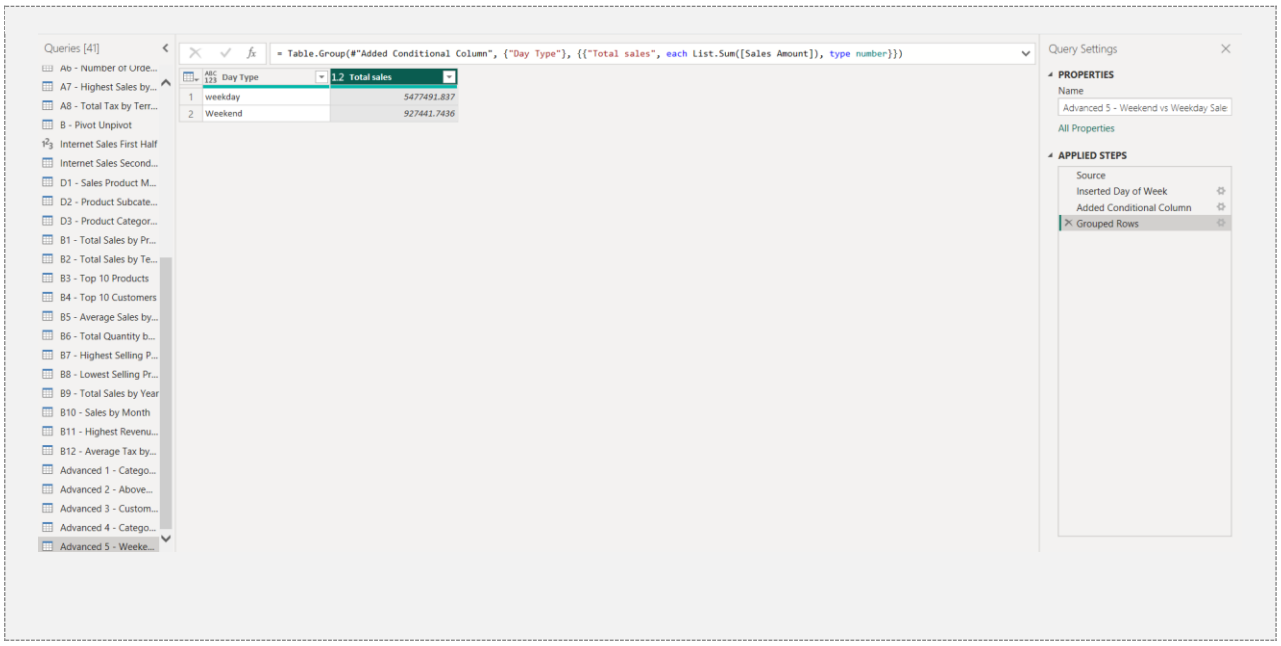
1. Write a DAX measure: % of Total Sales = DIVIDE([Total Sales], CALCULATE([Total Sales], ALL(Category))).
2. Place Category and % of Total Sales in a Table or Donut chart.
3. Format the measure as a percentage.

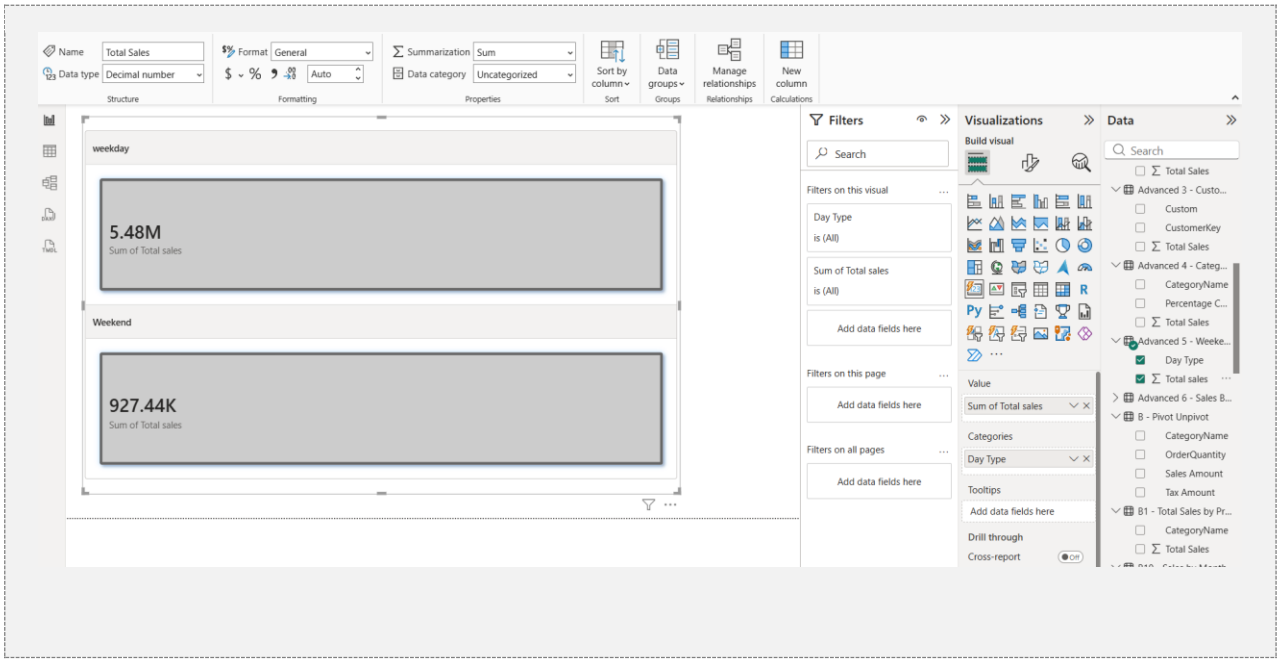




5. Compare weekend vs weekday sales by extracting the day of the week from the order date.
Flagging weekend vs weekday from the date lets you test whether purchasing behaviour differs by day type — a common retail analysis question.

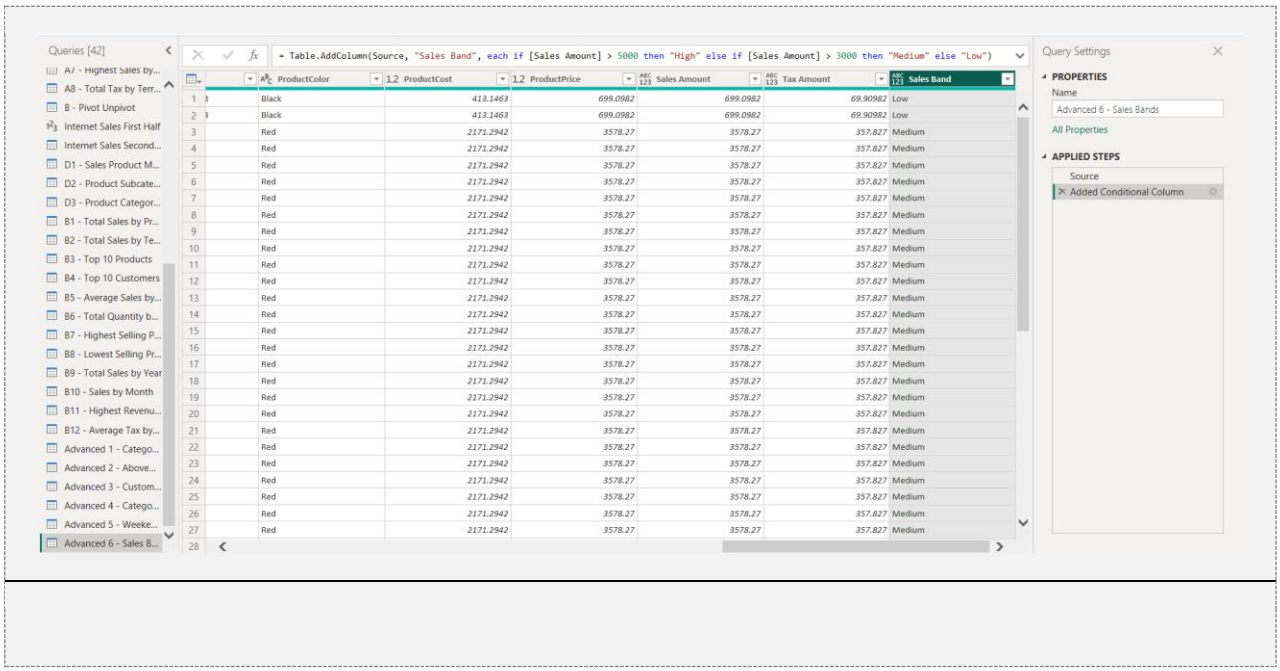
1. Add Column > Date > Day > Day of Week (or use a calculated column: Date.DayOfWeekName([OrderDate])).
2. Add a Conditional Column: 'Weekend' if the day is Saturday/Sunday, else 'Weekday'.
3. Place the Weekend/Weekday flag on a Bar chart with Total Sales as the value.

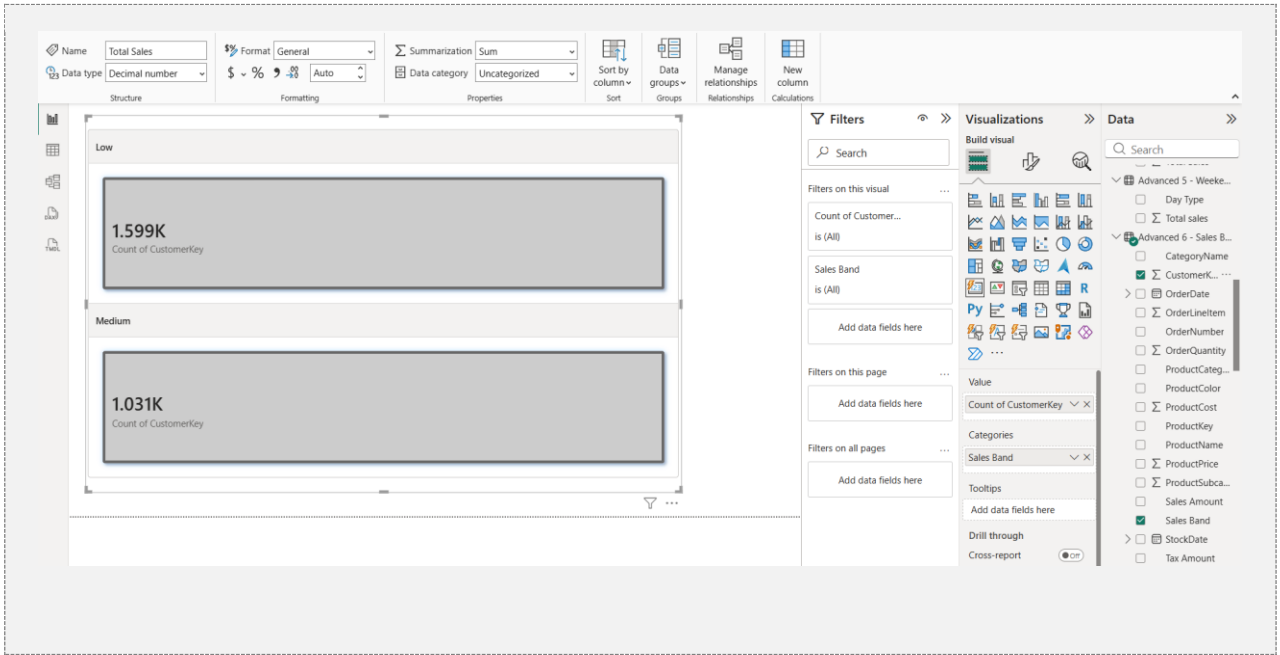




6. Create sales bands (Low, Medium, High) based on Sales Amount using Conditional Columns.
Unlike ADV3 (which bands total customer spend), this bands each individual transaction's Sales Amount — useful for a row-level 'deal size' filter on the dashboard.

1. Add Column > Conditional Column on Sales Amount.
2. Set thresholds: Low if <= Rs 14.97, Medium if <= Rs 159.00, else High (quartile-based cutoffs).
3. Name the new column 'Sales Band'.





Conclusion

This practical built on the earlier Power Query work by focusing on reshaping (pivot/unpivot), combining (append/merge) and analysing (DAX measures, ranking, conditional bands) the Adventure Works dataset end to end. Working through Grouping, Pivoting, Appending and Merging in sequence made it clear how each operation solves a different data-shaping problem, while the Sales Analysis and Advanced sections showed how the same clean working table can support many different business questions just by changing the measure or the grouping field.

Note: Reference values quoted under each 'Card / Dashboard Output' box were computed directly from the AdventureWorks Raw Data CSVs, so they can be used to check each screenshot's output before it is pasted in. Tax Amount assumes the standard 8%-of-Sales-Amount convention used in this tutorial dataset.